
LLM-DRIVEN DESIGN OF PHYSICS-CONSTRAINED CONSTITUTIVE MODELS: TWO AGENTS ARE BETTER THAN ONE

Marius Tacke¹, Matthias Busch², Kian Abdolazizi², Jonas Eichinger¹, Kevin Linka³, Roland Aydin^{2,4,5},
Christian Cyron^{1,2}

¹Helmholtz-Zentrum Hereon, Geesthacht, Germany

²Hamburg University of Technology, Hamburg, Germany

³RWTH Aachen University, Aachen, Germany

⁴Saarland University, Saarbrücken, Germany

⁵German Center for Artificial Intelligence, Kaiserslautern, Germany

ABSTRACT

Developing constitutive models that capture how materials deform under load traditionally requires years of specialized expertise in continuum mechanics, machine learning, and scientific programming. Large language models (LLMs) have recently been shown to lower this barrier by generating constitutive models on demand, but existing single-agent pipelines lack systematic checks that the resulting models respect fundamental physical laws. To close this gap, we introduce the first multi-agent LLM-driven approach for constitutive model generation: a Creator agent proposes a model tailored to the data, while an Inspector agent critically audits each proposal against nine physical constraints and returns it for refinement whenever a violation is detected. We demonstrate this concept with constitutive artificial neural networks (CANNs) and benchmark it on brain tissue, experimental rubber, and synthetic rubber, using two different LLM backbones (Claude Opus 4.7 and Kimi K2.5). Adding the Inspector raises the share of exported models that truly satisfy all physical constraints from 91% to a perfect 100% for Opus and from 37% to 56% for Kimi, while preserving near-baseline accuracy and remarkable generalization to unseen loading paths. In combination, the generated models are physically valid, highly accurate, and extrapolate reliably beyond the training data — properties that together make them directly usable in practice. Separating generation from inspection thus turns LLM-driven constitutive modeling into a genuinely trustworthy process. The paradigm is deliberately technique-agnostic and scales automatically with advances in LLM capability, opening a promising path toward automated, physics-aware model discovery.

1 Introduction

Materials are typically characterized in the laboratory under relatively simple loading conditions, most often uniaxial and occasionally biaxial or shear. In service, however, they are subjected to far more complex loading. Constitutive models bridge this gap by providing a mathematical framework that is calibrated against simple experiments yet capable of predicting material response under more general loading conditions.

Traditionally, scientists have developed theory-driven constitutive approaches in which the mathematical form of the model was chosen a priori from continuum mechanics, thermodynamics, and physical intuition, and then adjusted by hand to reproduce a limited set of experiments Hooke (1678); Maxwell (1867); Boltzmann (1874); Prandtl (1925); Reuss (1930). For hyperelasticity, this philosophy led to strain-energy formulations for rubber-like materials. Rather than deriving behavior from polymer-chain statistics, investigators postulated scalar energy functions of strain invariants and differentiated them to obtain stresses Mooney (1940); Rivlin (1948b,a); Treloar (1944); Valanis & Landel (1967); Ogden (1972). Despite their elegance and physical grounding, such hand-crafted formulations are inherently limited: developing and validating a new model typically requires years of specialized analytical effort, and the resulting expressions are constrained in complexity by the

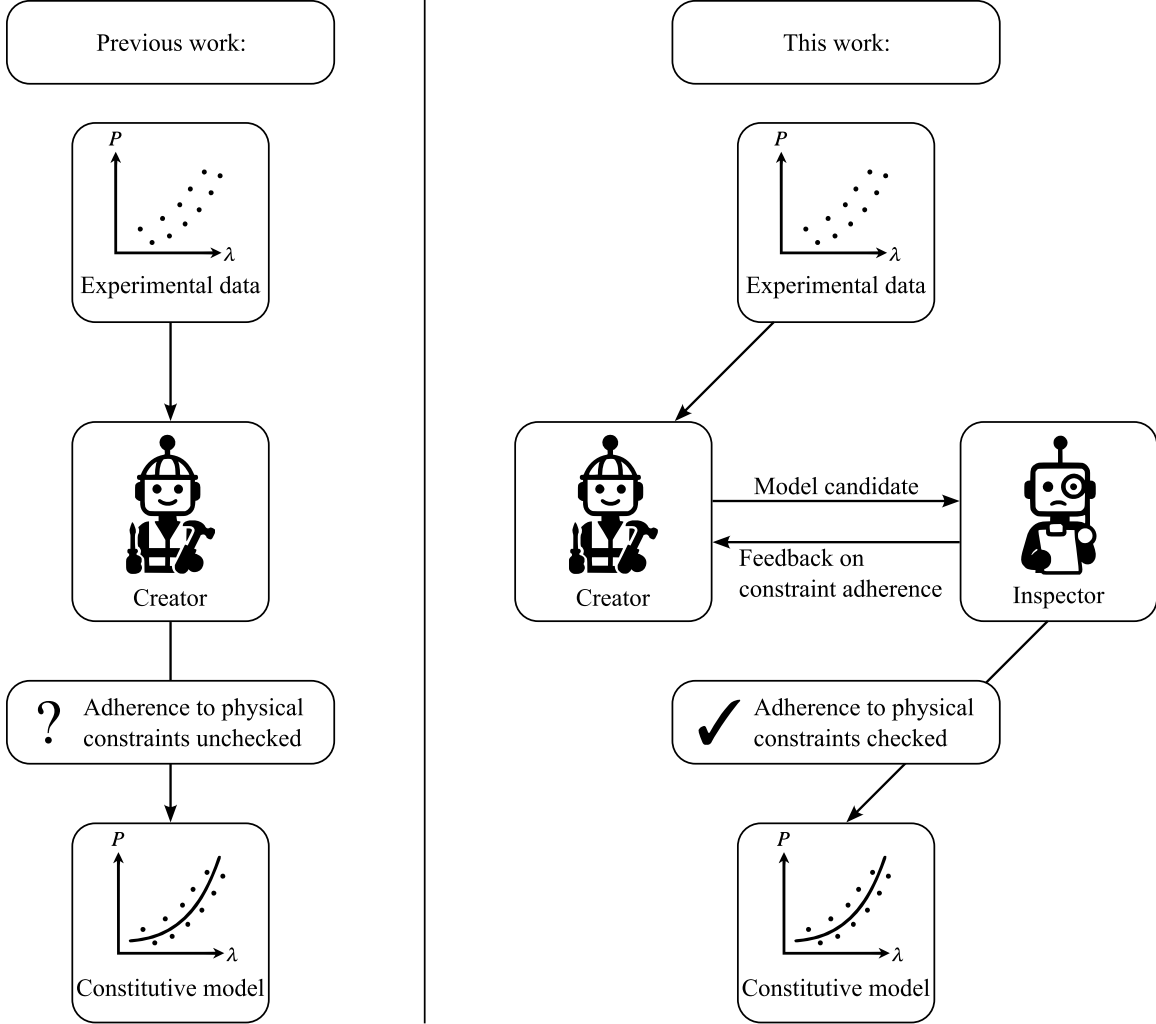


Figure 1: Previous LLM-based approaches to constitutive modeling rely on a single Creator agent that proposes a model and exports it directly (left). Instead, herein we propose a combination of two specialized agents (right): a Creator agent that proposes a constitutive model with a focus on physical admissibility, and an Inspector agent that critically audits the proposal against physical constraints. Only models that pass the Inspector are exported.

modeler’s ability to keep track of parameters, invariants, and their interacting effects—leaving little room to capture the full richness of real material behavior.

With the steady increase in computational power and the availability of larger experimental and simulated datasets, data-driven approaches have emerged as an alternative that bypasses the need for handcrafted functional forms altogether. Early efforts can broadly be characterized as black-box: spline interpolations of stress–strain data replace any explicit ansatz with piecewise polynomials and let the measurements speak directly through the interpolant Sussman & Bathe (2009); Latorre & Montáns (2013, 2014); Xu et al. (2015); distance-minimizing data-driven mechanics formulations reformulate boundary value problems as a search for admissible mechanical states closest to a material database, entirely circumventing the constitutive equation Kirchdoerfer & Ortiz (2016); Carrara et al. (2020); and, most influentially, artificial neural networks have been used as universal approximators of stress–strain mappings, starting with the seminal work of Ghaboussi et al. in the early 1990s Ghaboussi et al. (1991) and progressively refined through tighter integration into finite element solvers Hashash et al. (2004).

While these methods offer remarkable flexibility, they share three well-documented shortcomings: they extrapolate poorly beyond the training data and easily violate basic physical requirements such as objectivity,

material symmetry, or thermodynamic consistency, and the resulting models are essentially opaque, providing little insight into the underlying deformation mechanisms Linden et al. (2023); Fuhg et al. (2024); Hao et al. (2022); Abdolazizi et al. (2025).

To overcome these limitations, a third class of hybrid or grey-box models has emerged that combines the flexibility of neural networks with physical priors of varying strength, differing both in their architecture and in the level of interpretability they provide Fuhg et al. (2024); Hao et al. (2022). One line of work are Constitutive Artificial Neural Networks (CANNs), which embed strain invariants and admissible functional building blocks directly into the network topology so that the resulting strain energy is, by construction, objective, materially symmetric, polyconvex and thermodynamically consistent, and which have been extended from hyperelasticity to viscoelasticity, inelasticity and growth–remodeling and applied to a wide range of soft materials including brain tissue, skin, meat and rubber-like elastomers Linka et al. (2021); Linka & Kuhl (2023); Pierre et al. (2023a); Abdolazizi et al. (2024); Holthausen et al. (2024, 2025b,a); Linka et al. (2023a,b); Pierre et al. (2023b); Dal et al. (2023); Ehret & Stracuzzi (2022); Wiesheier et al. (2024); Peirlinck et al. (2025); McCulloch et al. (2024). A closely related line of work are Physics-Augmented Neural Networks (PANNs), which target the same set of constitutive conditions but employ somewhat denser, multilayered input-convex architectures combined with analytical correction terms, offering additional expressive flexibility Linden et al. (2023); Klein et al. (2022); As’ad et al. (2022); Kalina et al. (2025); Friedrichs et al. (2026); Geuken et al. (2026); Taç et al. (2022b,a, 2023). A complementary family of grey-box approaches relies on sparse regression and symbolic discovery to identify compact analytical laws from libraries of physically motivated terms, ranging from genetic programming Koza (1993); Abdusalamov et al. (2023) and sparse identification of nonlinear dynamics Brunton et al. (2016) to the EUCLID family that exploits full-field measurements and equilibrium constraints Flaschel et al. (2021, 2022); Joshi et al. (2022); Thakolkaran et al. (2022). More recent work explores spline-based Kolmogorov–Arnold Networks (KANs) Kolmogorov (1956); Liu et al. (2024c,b), which have been transferred to constitutive modeling as Constitutive KANs that combine the predictive flexibility of neural networks with symbolic post-processing for enhanced interpretability Abdolazizi et al. (2025); Ji et al. (2026). In a related spline-based approach, B-spline constitutive neural networks recover the strain energy of incompressible isotropic materials from tension–compression data Lee & Bathe (2025). Further hybrid concepts include model–data augmentation strategies Tushar et al. (2025) and physics-informed couplings to phase-field fracture Dammaß et al. (2025).

In the present work, we build on the family of neural-network-based approaches, in particular CANNs and PANNs. A central concern shared by all of them is the rigorous enforcement of the mathematical and physical conditions that any admissible hyperelastic constitutive law must satisfy: thermodynamic consistency, stress tensor symmetry, objectivity, material symmetry, polyconvexity (and the implied ellipticity that guarantees material stability), the volumetric growth condition, normalization of energy and stress in the reference configuration, and non-negativity of the strain energy Neff et al. (2015); Zee & Sternberg (1983); Upadhyay et al. (2019); Holzapfel (2001). A particularly systematic treatment is provided by Linden et al. Linden et al. (2023), who derive a network architecture that fulfills all of these conditions in an exact, by-construction manner for compressible isotropic and transversely isotropic hyperelasticity, building on architectural ingredients such as input-convex networks for polyconvexity Klein et al. (2022); As’ad et al. (2022); Geuken et al. (2026), analytical growth and normalization terms Kalina et al. (2025), stresses computed as gradients of a learned potential for thermodynamic consistency González et al. (2019); Linka & Kuhl (2023); Vlassis & Sun (2021), generalized structure tensors for anisotropy Kalina et al. (2025), and partially monotonic spline activations Fritsch & Carlson (1980); Polo-Molina et al. (2025) for KAN-based formulations Abdolazizi et al. (2025).

While the approaches reviewed above continue to automate ever larger parts of the constitutive modeling pipeline, their effective use still demands substantial expertise in continuum mechanics, machine learning, and scientific programming. Large language models (LLMs) have recently emerged as a complementary paradigm that further lowers this barrier by synthesizing domain knowledge, proposing hypotheses, and generating executable artifacts from natural-language descriptions Nejjar et al. (2025); Wysocki et al. (2024); Liu et al. (2025). One line of work leverages LLMs for the discovery of symbolic scientific laws, where the model proposes equation skeletons that are then calibrated and iteratively refined through evolutionary search, retrieval augmentation, self-reflection, chain-of-thought prompting, or LLM-based plausibility scoring Shojaaee et al. (2025); Wang et al. (2025a); Zhang et al. (2025); Sharlin & Josephson (2024); Taskin et al. (2026), with analogous strategies transferred to the discovery of simulation-ready constitutive laws Wu et al. (2026). A broader thrust employs LLMs as code generators that assemble full simulation or optimization pipelines from plain-language specifications, spanning surrogate modeling for engineering optimization Rios et al. (2024); Hao et al. (2024), PDE solvers and physics-informed surrogates Wuwu et al. (2025); Li et al. (2025), graph algorithms and materials-property prediction Verma et al. (2025); Huang et al. (2025), chemical-process

modeling Heyer et al. (2025), molecular dynamics Shi et al. (2025), CFD Pandey et al. (2025), and, in solid mechanics, the generation of UMAT subroutines for rate-dependent plasticity Gu et al. (2026). Beyond pure code generation, multimodal and tool-using systems couple LLMs with specialized numerical or experimental back ends for materials-property prediction, accelerated materials discovery, and autonomous experimentation Tang et al. (2026); Boyar et al. (2025); Bazgir et al. (2025); Boiko et al. (2023); Jia et al. (2024).

A recurring pattern in this literature is a bilevel structure in which an LLM-driven outer loop proposes candidates, while an inner numerical loop calibrates and evaluates them Chen et al. (2024a); Pandey et al. (2025). The scientific generative agent (SGA) generalizes this idea to open-ended hypothesis generation Ma et al. (2024), and the constitutive scientific generative agent (CSGA) specializes it to constitutive modeling by injecting assumptions such as isotropy, incompressibility, and an invariant basis Tacke et al. (2025b). On stress–strain benchmarks, CSGA outperforms SGA but remains clearly less accurate than purpose-built CANNs Tacke et al. (2025b). Most recently, Tacke et al. (2025a) bridged this gap by letting an LLM generate task-specific CANNs (GenCANNs) on demand, reaching accuracy on par with, and in several cases exceeding, human-designed CANNs. However, their framework leaves open whether the LLM-generated networks actually satisfy the physical constraints—thermodynamic consistency, objectivity, material symmetry, polyconvexity, growth, normalization, and non-negativity of the strain energy—that hand-crafted CANNs fulfill by construction (cf. the discussion above and Linden et al. (2023)).

A parallel strand of the LLM literature departs from the monolithic single-model pipeline and instead distributes the task across specialized LLM instances that communicate, critique, and refine each other’s outputs. The empirical case rests on two observations: structured debate and self-consistency among multiple LLMs consistently improve factuality and reasoning over single-model chain-of-thought Du et al. (2024); Gridach et al. (2025), and rigid role assignments can be replaced by dynamic task decomposition, automatic agent instantiation, and evolutionary restructuring of agent teams Wang et al. (2025b); Chen et al. (2024b); Tang et al. (2025); Liu et al. (2024a); Nettem et al. (2025); Yuan et al. (2025). A recurring design principle is the contrast between generative agents, which propose code, hypotheses, or candidate solutions, and inspecting agents, which critique, verify, and refine these outputs Gridach et al. (2025); Chen et al. (2024b); Liu et al. (2024a).

This paradigm has been embraced by the scientific-discovery community, where agent teams now orchestrate large portions of the research loop — from hypothesis generation and equation discovery through experiment design, simulation, and manuscript drafting — and consistently outperform strong single-LLM baselines on novelty, symbolic accuracy, and end-to-end autonomy Lu et al. (2024); Swanson et al. (2025); Boiko et al. (2023); Su et al. (2025); Yang et al. (2025); Xia et al. (2025); Hu et al. (2026); Yang et al. (2026). In materials and mechanics, where reasoning must combine heterogeneous data, physical laws, and numerical tools, multi-agent systems have been applied to bioinspired materials design, protein discovery, modular multi-task materials workflows, literature mining, robotic synthesis, earth-science data fusion, and, closest to the present setting, collaborative finite-element code generation for continuum mechanics Ghafarollahi & Buehler (2025, 2024); Chaudhari et al. (2026); Lin et al. (2025); Shi et al. (2026); Pantukhin et al. (2025); Ni & Buehler (2024). Across these studies, the multi-agent advantage is most pronounced whenever the task requires cross-disciplinary reasoning, iterative verification against external simulators, or reconciliation of conflicting information Ghafarollahi & Buehler (2025); Swanson et al. (2025); Su et al. (2025); Xia et al. (2025); Hu et al. (2026).

Despite this momentum, no existing multi-agent system has been tailored to the generation of physics-constrained constitutive neural networks: CSGA and GenCANN Tacke et al. (2025b,a) operate as essentially single-LLM pipelines and provide no checks on the physical admissibility of the resulting models. The present work closes this gap and makes three main contributions. We introduce the first multi-agent LLM architecture for constitutive model generation, in which a Creator proposes a model tailored to the data at hand and an Inspector critically audits each proposal against the physical constraints that any admissible hyperelastic model must satisfy: thermodynamic consistency, stress symmetry, objectivity, material symmetry, polyconvexity, growth, energy normalization, stress normalization, and non-negativity of the strain energy. Second, we show that this specialization is not merely a conceptual improvement but a measurable one. Across two LLM backbones and three benchmark datasets, adding the Inspector raises the share of exported models that truly satisfy all physical constraints from 91% to 100% for Claude Opus 4.7 and from 37% to 56% for Kimi K2.5, all while preserving near-baseline accuracy and strong generalization to unseen loading paths. Third, we deliberately formulate the paradigm in a technique-agnostic way: although we demonstrate it on CANNs, the same Creator–Inspector loop can be instantiated around PANNs, CKANs, or symbolic laws with only minor changes to the prompts, providing a general template for physics-aware, LLM-driven constitutive modeling that scales automatically with future progress in both agentic design and LLM capability.

2 Background

2.1 Continuum mechanics preliminaries

The constitutive models considered throughout this paper map a deformation state to a stress response, and the physical constraints that the LLM-generated CANNs are later required to satisfy are formulated in precisely these quantities. We therefore begin by fixing the kinematic and stress measures used in the sequel and by briefly recalling the invariant-based formulation of hyperelasticity, following the standard treatment of Holzapfel (2001).

Let $\mathbf{X}$ denote the position of a material point in the reference configuration and $\mathbf{x}$ its position in the current configuration. The local kinematics of the deformation are captured by the deformation gradient $\mathbf{F}$, from which the right Cauchy-Green deformation tensor $\mathbf{C}$ follows as

$$\mathbf{F} = \frac{\partial \mathbf{x}}{\partial \mathbf{X}}, \quad \mathbf{C} = \mathbf{F}^T \mathbf{F}.$$

For homogeneous loading scenarios, the deformation can often be parameterized by a single scalar. In uniaxial tension, this scalar is the stretch $\lambda = l/l_0$, i.e., the ratio between the current length l and the reference length l_0 , which appears as the corresponding diagonal component of $\mathbf{F}$. In simple shear, adjacent material layers are displaced tangentially relative to one another, and the amount of shear $\gamma = u/h$, defined as the lateral displacement u divided by the specimen height h , enters $\mathbf{F}$ as the associated off-diagonal component. Pure shear, in turn, is realized as a planar deformation in which the specimen is stretched by λ along one in-plane direction while being constrained in the orthogonal in-plane direction, so that only the through-thickness direction contracts to preserve the volume.

The principal scalar invariants of $\mathbf{C}$ are

$$I_1 = \text{tr}(\mathbf{C}), \quad I_2 = \frac{1}{2} [\text{tr}(\mathbf{C})^2 - \text{tr}(\mathbf{C}^2)], \quad I_3 = \det(\mathbf{C}).$$

When the material is treated as incompressible, volume preservation implies $\det(\mathbf{C}) = 1$ and thus $I_3 = 1$, so that only I_1 and I_2 remain as independent isotropic invariants. Anisotropy can be accommodated by enriching this set with additional invariants associated with one or more preferred material directions. Transverse isotropy, for example, introduces a single preferred direction together with a corresponding structure tensor and yields two further pseudo-invariants that quantify the stretch and squared stretch along that direction. Orthotropy is handled analogously by adding a second, independent fiber family and its associated invariants. In the present work, however, we restrict ourselves to isotropic behavior for the sake of simplicity, such that the constitutive response depends exclusively on I_1 and I_2 .

A material is termed hyperelastic when its mechanical response can be derived from a scalar strain-energy density function Ψ . This class of models provides an accurate description of rubber-like materials as well as of numerous soft biological tissues, and it is the exclusive setting considered here. Constitutive modeling then amounts to specifying Ψ as a function f of the deformation, expressed through the invariants of $\mathbf{C}$. Once Ψ is prescribed, the isochoric contribution to the first Piola-Kirchhoff stress is obtained by differentiation, while the incompressibility constraint is enforced through a volumetric term $-p\mathbf{F}^{-T}$, in which the Lagrange multiplier p plays the role of a hydrostatic pressure,

$$\Psi = f(I_1, I_2), \quad \mathbf{P} = \frac{\partial \Psi}{\partial \mathbf{F}} - p\mathbf{F}^{-T}.$$

The first Piola-Kirchhoff stress $\mathbf{P}$ expresses the force acting in the current configuration per unit area of the reference configuration and therefore provides a natural work-conjugate pairing with $\mathbf{F}$, which is particularly convenient when experimental data are reported in terms of reference geometry. As a second-order tensor, $\mathbf{P}$ is represented in three dimensions by a 3×3 matrix, and we denote its component in row i and column j by P_{ij} .

2.2 Constitutive artificial neural networks (CANNs)

The central contribution of this work is a new paradigm for generating constitutive models through LLM-based agents, together with a mechanism to check that the generated models respect the physical constraints of hyperelasticity. This paradigm is deliberately formulated in a technique-agnostic way and is, in principle, compatible with any parameterized family of constitutive models. For the purpose of demonstration, however, a concrete choice has to be made, and throughout this paper we instantiate the paradigm with CANNs, which offer a favorable trade-off between flexibility and physical structure and are by now well established in the community.

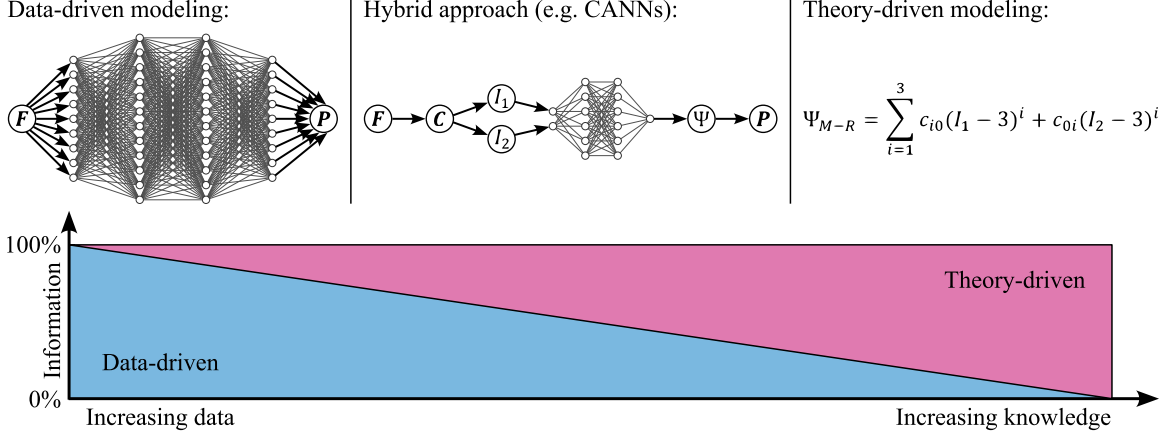


Figure 2: Spectrum of constitutive models for hyperelastic materials. Hand-crafted symbolic equations (right) can embed physical structure by design but their derivation is very time-consuming. By contrast, pure black-box neural networks mapping strain directly to stress (left) are easy to train and highly flexible but usually do not guarantee compliance with physical constraints. Hybrid approaches, such as constitutive artificial neural networks (CANNs) sit in the middle: they express a scalar strain-energy density Ψ as a function of invariants of $\mathbf{C}$ and recover the stress $\mathbf{P}$ by differentiation. This can be used to ensure compliance with certain concepts of physics. At the same time, they represent the functional relationship $\Psi = f(I_1, I_2)$ by a neural network so that they are easy and fast to obtain.

As illustrated in Figure 2, constitutive models for hyperelastic materials can broadly be placed on a spectrum ranging from fully hand-designed analytical laws, over hybrid or grey-box formulations, to pure black-box models that map strain directly to stress. CANNs are designed to combine the physical reliability of hand-crafted analytical laws with the expressive flexibility of black-box neural networks: they retain the invariant-based hyperelastic template introduced in Section 2.1, in which a scalar strain-energy density Ψ is expressed as a function of the invariants of $\mathbf{C}$ and the corresponding stress $\mathbf{P}$ is recovered by differentiation, but represent the functional relationship $\Psi = f(I_1, I_2)$ itself by a neural network. The network architecture is thereby chosen such that key physical requirements are satisfied by construction (Linka et al., 2021; Linden et al., 2023). Since physics-augmented neural networks (PANNs) share the same underlying philosophy, they could be substituted for CANNs within our paradigm at no additional conceptual cost.

2.3 Physical constraints and their validation

Hyperelastic constitutive models are subject to a well-established set of mathematical and physical conditions that any admissible model must satisfy. Following Linden et al. (2023), we consider nine such constraints: thermodynamic consistency, symmetry of the stress tensor, objectivity, material symmetry, polyconvexity, the volumetric growth condition, energy normalization, stress normalization, and non-negativity of the strain energy. The Creator is asked to enforce all of them by construction, the Inspector is asked to check that they are indeed enforced, and our numerical tools provide an external check whose verdicts we use as ground-truth labels in Section 4. With the exception of trivial cases such as evaluating the energy and stress at the reference configuration, these validators do not provide definite mathematical proofs: they are sampling-based procedures that probe each constraint at a finite but carefully chosen set of deformation states, rotations, and direction vectors, declaring a constraint as satisfied only when no violation is found across the entire sample. The samples are designed to cover the relevant spaces densely enough that a violating model is, with high likelihood, exposed; the detailed specifications are given in Appendix A.

Thermodynamic consistency A hyperelastic model must not create or destroy energy in purely elastic processes. The Clausius-Duhem inequality reduces, in this setting, to the requirement that the stress derives from a scalar strain energy potential Ψ as $\mathbf{P}_{\text{iso}} = \frac{\partial \Psi}{\partial \mathbf{F}}$ (Linden et al., 2023; González et al., 2019; Upadhyay et al., 2019). This potential structure is enforced by construction: a neural-network block predicts Ψ , the isochoric stress is obtained by automatic differentiation, and the volumetric contribution is added through a Lagrange-multiplier pressure p that enforces incompressibility. Since implementation-level mistakes can still break it in practice, the numerical validator tests path independence directly along closed deformation loops

and along open two-path comparisons. On closed loops, it checks that the cumulative work returns to zero relative to the total work expended and that the stress recovers its initial value upon loop closure, ruling out hidden statefulness. On open paths between the same end states, it checks that the two cumulative works agree with each other and that each of them reproduces the corresponding energy difference pointwise.

Symmetry of the stress tensor The balance of angular momentum requires the Cauchy stress σ to be symmetric, which in turn imposes a symmetry condition on the first Piola–Kirchhoff stress through $\sigma = J^{-1} \mathbf{P} \mathbf{F}^T$ (Holzapfel, 2001; Linden et al., 2023). This symmetry is automatically inherited when Ψ is expressed as a function of invariants of $\mathbf{C}$, rather than of the components of $\mathbf{F}$ or $\mathbf{C}$ directly. The validator computes σ from the model’s stress prediction at a representative set of deformation gradients and checks that $\sigma = \sigma^T$ holds within a small tolerance.

Objectivity A constitutive model must not depend on the choice of observer: superimposing a rigid rotation on the current configuration must leave the energy unchanged, $\Psi(\mathbf{Q}\mathbf{F}) = \Psi(\mathbf{F})$ for all $\mathbf{Q} \in \mathcal{SO}(3)$ (Linden et al., 2023; Holzapfel, 2001). Formulating Ψ in terms of invariants of $\mathbf{C} = \mathbf{F}^T \mathbf{F}$ enforces this property by construction. The validator probes it numerically by comparing $\Psi(\mathbf{F})$ to $\Psi(\mathbf{Q}\mathbf{F})$ across a representative set of deformations and rotations.

Material symmetry The energy must also be invariant under the symmetry transformations of the material itself, $\Psi(\mathbf{F}\mathbf{Q}^T) = \Psi(\mathbf{F})$ for all $\mathbf{Q}$ in the symmetry group of the material (Linden et al., 2023; Holzapfel, 2001). For the isotropic incompressible materials considered here, the symmetry group is $\mathcal{O}(3)$, and the same invariant-based formulation that ensures objectivity also ensures material symmetry. The validator is structurally analogous to the objectivity check, but applies the rotation from the right and additionally includes reflections.

Polyconvexity (and ellipticity) Polyconvexity of Ψ is the standard route to material stability of the constitutive model, which in turn leads to favorable behavior in numerical applications: polyconvexity implies quasiconvexity, which in turn implies rank-one convexity, also known as ellipticity (Neff et al., 2015; Zee & Sternberg, 1983; Linden et al., 2023). Polyconvexity is much easier to enforce architecturally than ellipticity, which is why CANNs are typically built around it, and why we ask the Creator to enforce it by construction. For the numerical validation, we follow Linden et al. (2023) and check the weaker but practically relevant ellipticity condition directly, namely that the rank-one form $(\mathbf{a} \otimes \mathbf{b}) : (\partial^2 \Psi / \partial \mathbf{F} \partial \mathbf{F}) : (\mathbf{a} \otimes \mathbf{b})$ is non-negative for all direction pairs $(\mathbf{a}, \mathbf{b})$ at all admissible deformations. The validator samples this form on a grid of diagonal deformation gradients and a Fibonacci-distributed set of direction pairs.

Growth condition The model should respond with infinite energy when the volume tends to zero or infinity, $\Psi \rightarrow \infty$ as $J \rightarrow 0^+$ or $J \rightarrow \infty$ (Linden et al., 2023; Holzapfel, 2001). Under the incompressibility constraint $J = 1$, only deformation states satisfying this constraint are admissible, so the growth condition is trivially satisfied; the validator returns success without any computation.

Energy normalization The strain energy must vanish in the reference configuration, $\Psi(\mathbf{F} = \mathbf{I}) = 0$ (Linden et al., 2023). This is enforced architecturally by subtracting the network output evaluated at $\mathbf{C} = \mathbf{I}$ from the predicted energy. The validator simply evaluates the model at $\mathbf{F} = \mathbf{I}$ and checks that the returned energy lies within a small tolerance of zero.

Stress normalization In the reference configuration, the material must be stress-free, $\mathbf{P}(\mathbf{F} = \mathbf{I}) = \mathbf{0}$ (Linden et al., 2023). Under isotropic incompressibility, this is automatically guaranteed by the Lagrange-multiplier pressure p that enforces incompressibility. The validator evaluates the predicted stress at $\mathbf{F} = \mathbf{I}$ and verifies that all of its entries are below a small tolerance.

Non-negativity of the strain energy Finally, the strain energy must be non-negative for every admissible deformation, $\Psi(\mathbf{F}) \geq 0$ (Linden et al., 2023). In our setting, this property is automatically inherited from the combination of polyconvexity, energy normalization, and stress normalization. The validator nevertheless checks it explicitly by evaluating Ψ across the same representative set of deformation gradients used for the other checks and verifying that no negative value appears.

3 Method

The central motivation of this work is to lower the barrier of accessibility to constitutive modeling. As the preceding sections and the extensive literature on the subject make clear (Linden et al., 2023; Holzapfel, 2001; Neff et al., 2015; Linka et al., 2021; Linka & Kuhl, 2023), developing an effective constitutive model is far from trivial: it requires a working knowledge of continuum mechanics, a careful choice of functional building blocks, and a practical implementation that respects a substantial list of physical constraints. Recent advances in large language models (LLMs) have shown impressive capabilities in scientific discovery and code generation at a level of usability that was unthinkable only a few years ago (Nejjar et al., 2025; Wysocki et al., 2024). It is therefore natural to ask whether they can also be leveraged to carry part of the constitutive modeling burden.

Our approach follows and extends a short but rapidly evolving line of research. The scientific generative agent (SGA) of Ma et al. (2024) was the first to propose a general bilevel framework in which an LLM generates symbolic scientific hypotheses in the outer loop, while a differentiable simulator calibrates their continuous parameters in the inner loop. SGA was demonstrated on constitutive law discovery from motion observations and on molecular design, and consistently outperformed pure chain-of-thought or single-loop LLM baselines.

The constitutive scientific generative agent (CSGA) of Tacke et al. (2025b) specialized SGA for hyperelastic constitutive modeling. It injects domain knowledge into the prompt—such as isotropy, incompressibility, an invariant- or principal-stretch-based functional basis, and the requirement that the stress vanishes in the reference configuration—and replaces the expensive forward simulation with a direct regression on tabulated stress-strain data. CSGA clearly outperforms SGA on standard benchmarks but remains noticeably less accurate than purpose-built CANNs.

Most recently, Tacke et al. (2025a) closed this accuracy gap with GenCANN, in which the LLM no longer proposes a symbolic law but instead generates, on demand, a full CANN tailored to the material and dataset at hand. The generated networks match or exceed the accuracy of human-designed CANNs while retaining a plain-text interface. What GenCANN does not provide, however, is any systematic check that the generated networks satisfy the physical constraints listed in Section 2.3. This is precisely the gap that we close in the present work.

3.1 Multi-agent architecture: Creator and Inspector

Rather than proposing yet another single-agent pipeline, we introduce a more general paradigm that we believe is a natural next step for LLM-driven constitutive modeling: two complementary agents that together pursue a common goal. A *Creator* focuses on proposing an expressive and well-fitting constitutive model, while an *Inspector* critically audits the Creator’s proposal against the physical constraints discussed in Section 2.3. This contrast between a generator and a critic is a classical divide-and-conquer strategy - it underlies the entire architecture of generative adversarial networks, and has recently been shown to be particularly effective in multi-agent LLM systems, where a clean separation between generating and inspecting agents consistently improves factual correctness and reasoning over single-model pipelines (Du et al., 2024; Gridach et al., 2025; Chen et al., 2024b; Liu et al., 2024a). Figure 1 illustrates this shift from the single-creator pipelines of earlier work to the Creator-Inspector paradigm adopted here.

Importantly, the paradigm itself is independent of the specific family of constitutive models at its core. The Creator could, in principle, generate symbolic laws, PANNs, CKANs, or any other parameterized constitutive model, and the Inspector would remain responsible for checking the same physical constraints. We deliberately speak of models rather than CANNs to emphasize this generality; our use of CANNs in the experiments that follow is a concrete and well-established demonstration vehicle, not a defining feature of the approach.

3.2 Pipeline

The complete workflow is illustrated in Figure 3 and proceeds as follows.

The Creator is prompted to implement a constitutive model for the given material class and dataset. The prompt briefly lists the nine physical constraints of Section 2.3, without reproducing their full mathematical treatment, and requests a concrete Python implementation. The Creator responds with a complete model in its reply, from which the code is extracted automatically.

Before any inspection takes place, the proposed model is subjected to a set of automated syntactic checks. We verify that the Python code is executable, that the model can be instantiated, trained on the provided data, used

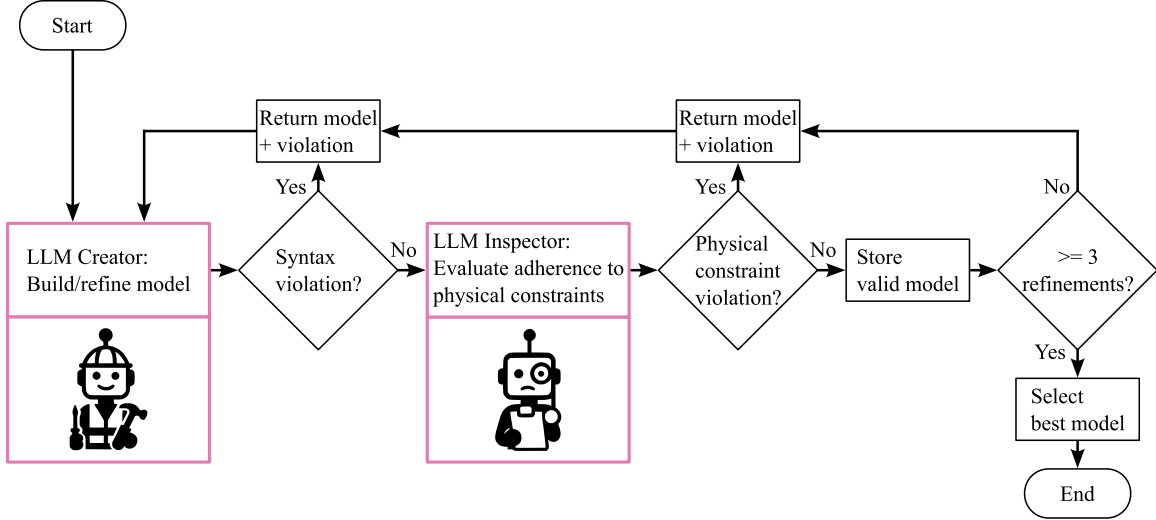


Figure 3: Detailed workflow of the Creator-Inspector pipeline. The Creator is prompted to implement a constitutive model and returns a Python implementation. The implementation is first subjected to automated syntactic checks (executability, training, prediction, save/load); failures are returned to the Creator for correction. Valid implementations are forwarded to the Inspector, which audits them against the nine physical constraints discussed in Section 2.3 and returns a verdict per constraint. Flagged violations are sent back to the Creator with the Inspector’s justifications; fully approved models are exported. Once a valid model is obtained, the pair is invited to refine it twice based on training-data predictions and R^2 /MSE metrics, so that each full run yields three inspector-approved models, of which the most accurate is selected.

for prediction, and correctly saved and loaded. Some failures are caused by outright syntax errors from the LLM; others by subtle incompatibilities between the LLM-generated model and the surrounding static code used for training, evaluation, and serialization. Whenever one of these checks fails, the model is returned to the Creator with a concise description of the failed check and a request for correction. Only implementations that pass all syntactic checks are forwarded to the Inspector.

The Inspector is then prompted with the full Python script of the model and with the same list of nine physical constraints, enriched by short practical hints on how each constraint can be enforced architecturally (for example, by using an input-convex subnetwork for polyconvexity or by deriving stresses from a scalar potential for thermodynamic consistency). The Inspector is required to return its verdict stating, for every constraint, whether it is fulfilled or violated, together with a short justification.

If the Inspector flags at least one constraint as violated, the model is sent back to the Creator together with the list of flagged constraints and the Inspector’s justifications. The Creator is then asked to modify its implementation so as to address these concerns. If all constraints are flagged as fulfilled, the implementation is stored and exported as a valid model.

Once a valid model is obtained, the Creator-Inspector pair is asked to refine it. The current implementation is passed back to the Creator, together with a tabular summary of its predictions on the training data and the corresponding R^2 and MSE values, and the Creator is invited to propose an improved variant. Each refined model again passes through the syntactic checks and the Inspector before being exported. The refinement loop is executed twice after the initial generation, so that each full run produces, in total, three inspector-approved models. The most accurate one is then selected as the output of the run.

3.3 Two natural ablations

Two questions arise naturally from this design, and we address both by means of dedicated ablations.

The first question concerns the value of specialization itself. Since GenCANN (Tacke et al., 2025a) already demonstrated that an LLM can generate accurate constitutive models in a largely single-agent fashion, it is not self-evident that an additional Inspector is worth its computational and conceptual cost. To probe this, we run a full ablation in which only the Creator is active and every syntactically valid model is exported without

further verification. Because our prompts differ slightly from those of GenCANN, this ablation also serves as a fair baseline for the Inspector’s added value within our setup.

The second question concerns the Inspector’s own limitations. In principle, any physical constraint could simply be checked by directly running the numerical validation tools of Section 2.3. Doing so unconditionally would, however, conflate the capabilities of the LLM with those of the validators and obscure what the LLMs themselves actually understand about hyperelastic admissibility. We therefore test a second variant in which the Inspector is additionally granted access to the numerical validation tools. All prompts remain unchanged, except for a single sentence informing the Inspector that these tools are available and instructing it to invoke them only when it is genuinely uncertain about a specific constraint. This setup probes whether the LLM can accurately estimate its own uncertainty and use external tools selectively and effectively, rather than as a substitute for its own reasoning.

3.4 LLM backbones

We instantiate the Creator and the Inspector with two LLM backbones and run all experiments for each of them. As a frontier proprietary model we use Anthropic’s Claude Opus 4.7, and as an open-source counterpart we use MoonshotAI’s Kimi K2.5. Comparing the two allows us to assess how robust our findings are with respect to the choice of backbone, and to what extent the observed behaviors are specific to a particular model family.

3.5 Benchmark problems

To keep the focus on the LLM-powered agentic setup itself, we deliberately restrict our benchmark problems to hyperelastic, isotropic, and incompressible materials. Within this class, we select three datasets that are well established in the constitutive modeling literature and that together serve complementary purposes: a brain-tissue dataset that is particularly challenging to fit, a synthetic rubber dataset that permits the computation of ground-truth stresses for arbitrary deformations and thereby a clean assessment of generalization and extrapolation, and an experimental rubber dataset that anchors the synthetic one in physical reality.

The brain-tissue dataset captures the mechanical response of human cerebral tissue (Budday et al., 2017a,b, 2019), which is known to be soft, nearly incompressible, strain-stiffening, and asymmetric in tension and compression. Accurate models of it are relevant for impact simulation, injury prediction, and protective design. We use the data reported by Budday et al. (2017a), who performed mechanical tests on specimens excised from ten post-mortem human brains within 60 hours of death. Of the anatomical regions sampled, we consider the cortical gray matter. Three loading modes were applied, namely uniaxial tension, uniaxial compression, and simple shear, each in loading–unloading cycles, and the mean stress across the hysteresis loop was retained as the effective elastic response. Only 17 stress-strain points are reported per loading mode, which poses a genuine challenge for models expected to generalize well. Such sparsity is, however, characteristic of soft-tissue experiments and reflects standard practice in the field.

The two remaining datasets concern rubber, the prototypical hyperelastic solid, which sustains large and fully reversible strains far beyond the regime of linear elasticity and whose accurate modeling underpins the design of engineering components such as tires and seals. We rely on two complementary sources: the classic experimental dataset of Treloar (1944) and a synthetic counterpart introduced alongside the original CANN paper (Linka et al., 2021), which represents a similar but fictitious rubber-like material. Both cover the same three loading protocols, namely uniaxial tension, equibiaxial tension, and pure shear, with 15 samples per protocol, and we adopt the train-test split of the original CANN publication. Since the underlying material of the synthetic dataset is known in closed form, exact stresses can be computed for arbitrary deformations, which enables a controlled evaluation of how well a trained model generalizes beyond the loading paths seen during training.

4 Results

We organize the presentation of our results around a set of research questions that arise naturally from the methodology introduced above.

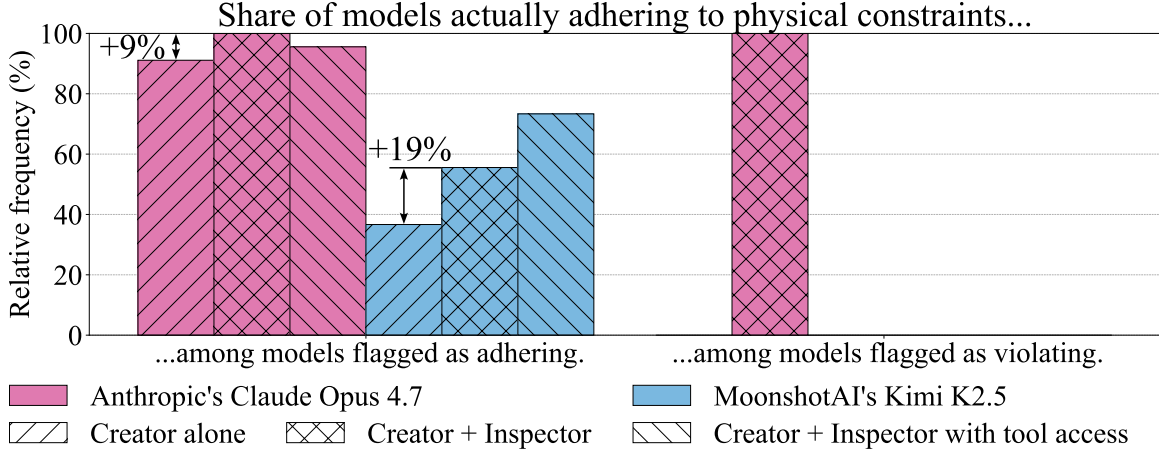


Figure 4: Adherence of generated models to physical constraints, split by Inspector verdict. For each configuration — Creator alone, Creator with Inspector, and Creator with tool-enabled Inspector, each powered by Claude Opus 4.7 or Kimi K2.5 — we report the fraction of models that truly adhere among those the Inspector flagged as adhering (left) and as violating (right). Since the Creator-alone setup has no Inspector, all its models are implicitly counted as flagged adhering. True adherence is established via the numerical validation procedure described in Section 2.3.

4.1 Does the specialization into a multi-agent system bring any advantage over the Creator alone?

The central claim of this work is that splitting the task of constitutive model generation between two specialized agents — the Creator and the Inspector — enables automatic and critical validation of the proposed models, and in doing so makes the overall system more trustworthy than a single LLM acting on its own. Whether this claim holds translates into a concrete question: how often does each approach actually produce models that respect the underlying physical constraints?

To answer this, we need a reliable notion of ground truth. We obtain it by numerically validating every proposed model using the procedure described in Section 2.3, and we consider a model to truly adhere to the physical constraints if and only if it passes these checks. This must not be confused with the Inspector’s verdict, which only determines how a model is flagged: models judged acceptable by the Inspector are exported as the outcome of the refinement round and labeled flagged as adhering, while from those flagged as violating we export a random subset so that we can also assess how accurate the Inspector’s negative verdicts are.

Pooled across 10 runs on three benchmark problems with three refinement rounds per run, Figure 4 reports the relative frequency of models that truly adhere among those the Inspector flagged as adhering (left) and as violating (right). From a practical standpoint, the most informative quantity is the share of flagged-as-adhering models that truly adhere, as it directly reflects how much a user can trust a model returned by the system. For Claude Opus 4.7, the Creator alone reaches 91%, and adding the Inspector lifts this to 100%. For Kimi K2.5, the corresponding numbers are 37% without and 56% with the Inspector. Two observations stand out. First, even between frontier LLMs the absolute performance differs substantially, and with Claude Opus 4.7 the proposed approach becomes fully reliable for this problem setting. Second — and this is the answer to the question posed in this subsection — the gains contributed by the Inspector, 9 and 19 percentage points, are consistent across both backbones and show that the second, specialized agent meaningfully increases our confidence in the physical validity of the generated models.

4.2 How reliable is the Inspector’s verdict?

So far we have compared the Creator alone against the Creator–Inspector pair. As described in Section 3.3, we additionally evaluated a third variant in which the Inspector has access to the numerical validation tools used to check adherence to the physical constraints. Figure 4 reports the relative frequency of truly adhering models among those flagged as adhering (left) and as violating (right), for all three configurations and for both Claude Opus 4.7 and Kimi K2.5.

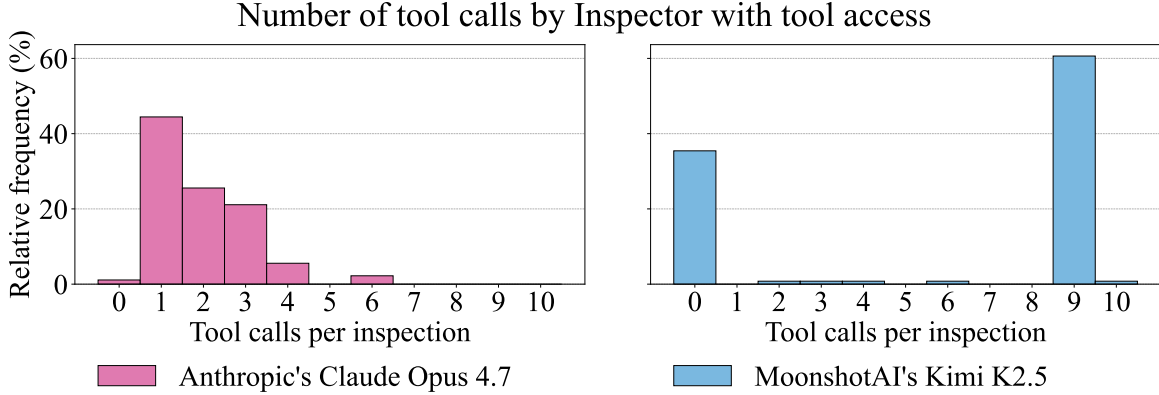


Figure 5: Tool usage of the Inspector per inspection. Distribution of the number of tools called by the Inspector in a single inspection, shown separately for Claude Opus 4.7 and Kimi K2.5. The Inspector is instructed to invoke tools only when uncertain. Opus follows this instruction closely and predominantly calls just one or two tools, whereas Kimi exhibits a bimodal pattern, either calling all available tools or none at all.

Looking at the flagged-as-adhering side first, one would expect the share of truly adhering models to grow as the pipeline becomes more elaborate, and especially when the Inspector is granted tool access. For Kimi this expectation is met: the share rises from 37% (Creator alone) to 56% (with Inspector) and further to 73% with tool access. For Opus, however, the trend breaks. The Inspector without tools already lifts the Creator from 91% to 100%, leaving no room for tool access to improve further — and in fact the share even drops slightly to 96%.

The flagged-as-violating side reveals a second, equally striking pattern. Of the models that the Opus Inspector (without tools) flags as violating, 100% in fact adhere to the physical constraints. In other words, when in doubt, Opus tends to err on the side of raising a flag — a conservative and, for our purposes, desirable behavior. With tool access, the Opus-powered Inspector does not flag any violations at all. The Kimi Inspector behaves in exactly the opposite way: every model it flags as violating does in fact violate, so its false-alarm rate is already zero without tools.

Figure 5 helps explain these patterns by showing how many tools the Inspector calls per inspection when tool access is available. The Inspector is instructed to invoke tools only when it is unsure, since we want to measure the LLMs’ own capabilities in creating and inspecting models rather than those of the numerical validators. Opus follows this instruction closely, most often calling just one or two tools to resolve specific doubts. Kimi, in contrast, exhibits a bimodal behavior: it either calls all available tools or none at all, with almost nothing in between.

Taken together, these observations paint a coherent picture of the two backbones. Kimi appears inherently submissive: without tools it readily accepts violating models, and it only flags a model as violating when it is very certain or when the tools explicitly confirm a problem. This explains both the low raw performance without tools and the perfect precision on the violation side. Opus, on the other hand, is naturally more critical, which is why the Inspector already delivers perfect performance without tools. Its surprising drop in performance under tool access is consistent with its sparse tool usage: Opus respects the "only-when-unsure" instruction, but in a handful of cases it is incorrectly confident about specific constraints and then does not consult the tools. Conversely, it sometimes doubts constraints that are fulfilled and then draws false confidence from the tools it calls to check them. We return to this point in Section 4.4.

4.3 Are the violations raised by the Inspector solved by the Creator?

To assess how effectively the two agents collaborate, we track the states that a Creator–Inspector pair can be in over the course of a refinement loop and report the relative frequencies of transitions between these states in Figure 6. We define four states: Start, New violation, Repeated violation, and Adherence. Since we are interested in the agents’ own behavior, the state is determined entirely by the Inspector’s verdict, not by the numerical ground truth: a violation is a violation flagged by the Inspector, and adherence is adherence flagged by the Inspector. For this analysis we pool the runs with and without tool access into a single Inspector category.

Creator + Inspector state transitions:

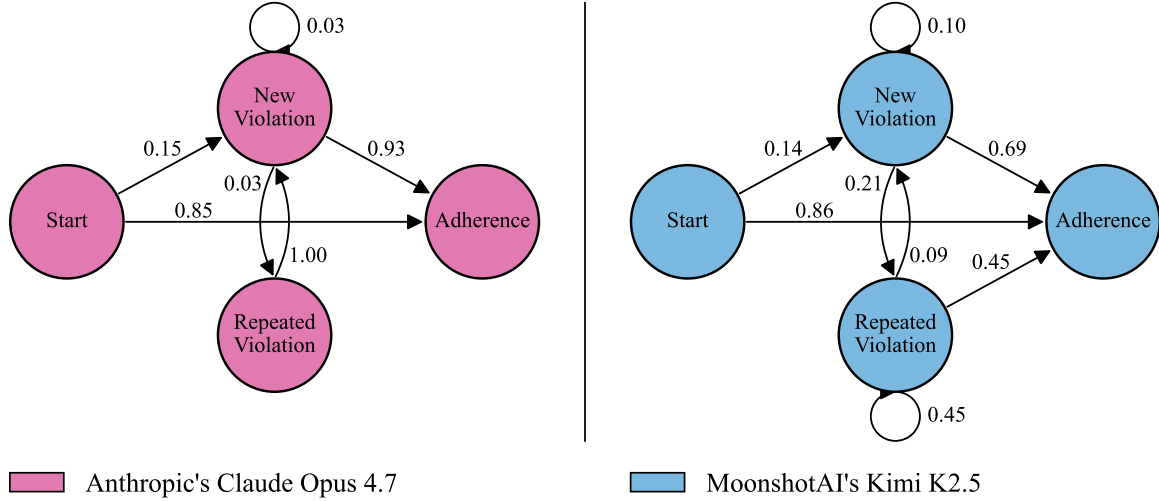


Figure 6: State transitions between Creator and Inspector across refinement rounds. Relative frequencies of transitions between the states Start, New violation, Repeated violation, and Adherence, shown separately for the pipelines powered by Claude Opus 4.7 and Kimi K2.5. States are defined solely by the Inspector’s verdict. Runs with and without tool access are pooled.

Effective collaboration would show itself in two ways: most models should reach adherence quickly, and when violations are raised they should rarely persist across iterations. For the Opus-powered pair, both criteria are clearly met. Already at the start, 85% of the Creator’s proposals are flagged as adhering to all physical constraints, leaving only 15% in the violation state. Of these, 93% are resolved directly in the next iteration and just 3% persist as repeated violations. Within this small remainder, none of the proposals is fully fixed after a further update; instead, each leads to a new violation. However, as these cases amount to only 3% of the initial 15%, they are clear outliers. The Kimi-powered pair starts from a comparable baseline — 86% adherence and 14% violations after the first proposal — but its behavior under refinement differs markedly. New violations are resolved in only 69% of the cases (compared to 93% for Opus), and 21% of them persist as repeated violations. Once a repeated violation is reached, the situation becomes even more difficult: 45% remain repeated violations in the next iteration as well. This indicates that the Kimi-powered Creator struggles to address the concerns raised by the Kimi-powered Inspector. On the upside, the other 45% of repeated violations are eventually resolved into full adherence rather than turning into new violations.

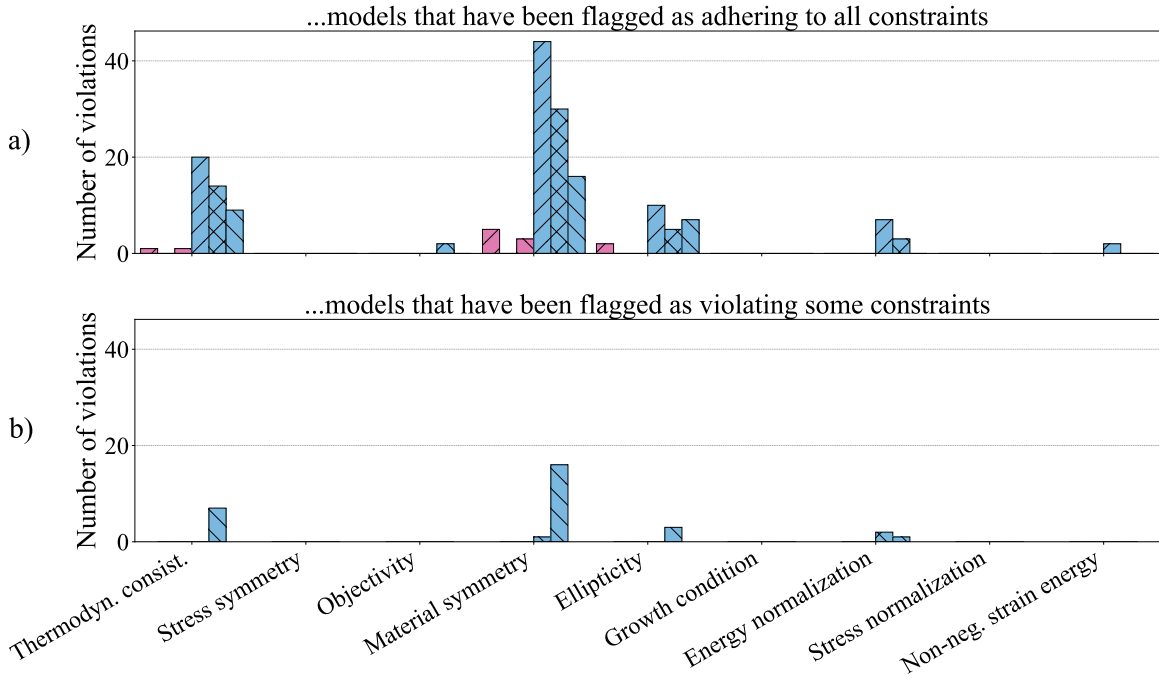
Overall, the Opus-powered Creator–Inspector pair collaborates effectively: violations raised by the Inspector are resolved in 93% of the cases, with the minor caveat that the few persistent ones tend to be replaced by new violations rather than fully resolved into adherence. The Kimi-powered pair collaborates noticeably less effectively, resolving only 69% of new violations directly and frequently getting stuck in cycles of repeated violations.

4.4 Zooming in: Which violations actually occur, which are detected, and which are missed?

Figure 7 breaks violations of the physical constraints down by type across four charts. Charts a) and b) show actual violations, as confirmed by the numerical validation of Section 2.3: Chart a) for models that the Inspector flagged as adhering to all constraints, and Chart b) for models that the Inspector flagged as violating some constraints. Charts c) and d) show flagged violations raised by the Inspector: Chart c) those that our numerical validation confirms to be correct, and Chart d) those that it does not. We use the same y-axis range in Charts a) and b) for direct comparability. The overall counts in b) are much lower than in a), because every refinement round exports exactly one Inspector-approved model, whereas violating models are only exported as a random subset for analysis; this is an artifact of our experimental design and does not affect the relative distributions across constraint types, which is what Figure 7 is about.

We first look at the actual violations in Charts a) and b). Among models flagged as adhering, the Opus-powered configurations produce very few violations overall and the Creator–Inspector pair without tools produces

Actual violations of physical constraints by...



Flagged violations of physical constraints that are...

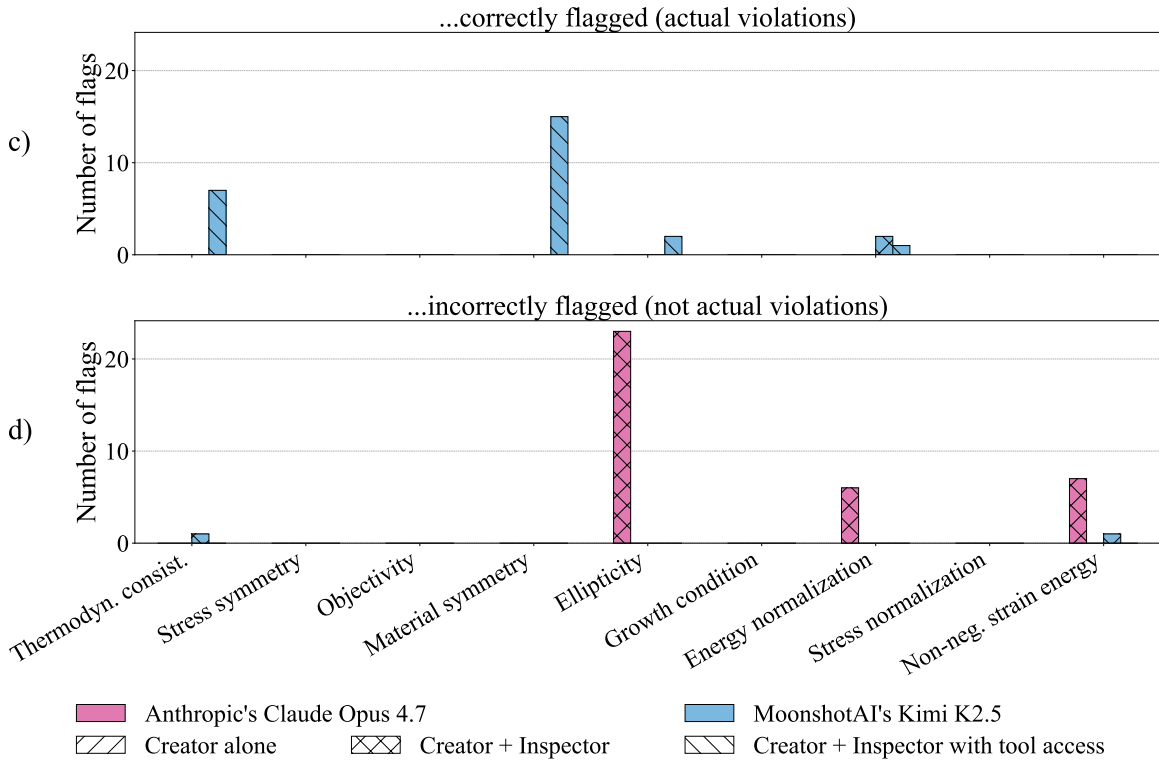


Figure 7: Distribution of physical constraint violations by type, separated into actual vs. flagged and correct vs. incorrect.

none, consistent with the 100% adherence reported in Section 4.1; the 4% that slip through the tool-enabled variant all stem from material symmetry, and the few violations of the Creator alone concern either material symmetry or ellipticity. The Kimi-powered configurations show a broader pattern dominated by material symmetry, followed by thermodynamic consistency, ellipticity, and energy normalization, with the overall count decreasing from the Creator alone to the Creator–Inspector pair and further to the tool-enabled variant. Among models flagged as violating, the Opus-powered configurations contribute almost nothing — the Creator alone implicitly flags every model as adhering, the Creator–Inspector pair’s flagged-as-violating models all turned out to truly adhere (cf. Section 4.2), and the tool-enabled variant did not raise any violation flag at all — whereas the Kimi distribution closely mirrors the one in Chart a), indicating that the constraints Kimi’s Creator struggles with are the same regardless of whether the Inspector later catches them.

The flagged violations in Charts c) and d) reflect the two characteristic behaviors identified in Section 4.2. Correctly flagged violations come almost exclusively from the tool-enabled Kimi Inspector, which is less a particular strength than a consequence of its bimodal tool usage: when it calls tools, it calls all of them, so most of the flagged violations have effectively been pre-confirmed by the same numerical checks we use as ground truth. Incorrectly flagged violations come almost exclusively from the Opus Inspector without tools, whose uncertainty is concentrated on ellipticity and, to a lesser extent, on energy normalization and non-negativity of strain energy; with tool access these doubts are resolved numerically and the unnecessary flags disappear.

Two takeaways stand out. For Opus, when in doubt — especially about ellipticity, energy normalization, and non-negativity of strain energy — the Inspector prefers to flag rather than to accept, which is inefficient but aligned with our goal of erring on the safe side. For Kimi, the violations that slip through are not caused by a blind spot for any single constraint but by small inaccuracies spread across the same constraints that the Kimi-powered Creator already finds hardest to satisfy.

4.5 How do the predictions of the generated models look in practice?

To get an impression of how the generated models behave in practice, we compare the predictions of models produced by our pipeline with established baselines from the literature. We focus on the setting with the Creator and Inspector but without tool access, and, from ten independent runs, select the most accurate model that also adheres to the physical constraints. This selection procedure reflects how we expect the pipeline to be used in practice: since it is built around stochastic LLMs, multiple runs should be performed and the best-performing model kept. To give the reader a concrete impression of the artifacts produced by the pipeline, the Python implementations of the two models selected for the experimental rubber dataset are shown in Appendix C.

Figure 8 shows the predictions for models generated with Claude Opus 4.7, Figure 9 those generated with Kimi K2.5, and Figure 10 the literature baselines. For brain tissue, we take the most accurate CANN reported across several optimization studies on this dataset Linka et al. (2023a); McCulloch et al. (2024); Pierre et al. (2023a); Budday et al. (2019) as baseline Pierre et al. (2023a). For both rubber datasets — the experimental and the synthetic one — we use the most accurate CANN from the original publication Linka et al. (2021). Charts a), b), and c) of Figures 8 - 10 show the predictions on the training loading conditions: uniaxial tension, uniaxial compression, and simple shear for the brain data, and uniaxial tension, equibiaxial tension, and pure shear for the rubber data.

The baselines achieve near-perfect predictions on all three datasets, confirming that highly accurate constitutive models are well established for these commonly used benchmarks. Our generated models match this performance on both rubber datasets, reaching near-perfect accuracy across all loading conditions. On the brain tissue data, however, they fall clearly behind. The Kimi-powered pair reaches R^2 values of 0.614 on tension, 0.651 on compression, and 0.829 on shear, while the Opus-powered pair reaches 0.453, 0.876, and 0.981, respectively. The baseline, in contrast, achieves 0.96, 0.99, and 1.00. It is worth noting that the brain tissue baseline is the product of multiple dedicated optimization studies Linka et al. (2023a); McCulloch et al. (2024); Pierre et al. (2023a); Budday et al. (2019), from which we selected the best reported accuracy, so the bar is set very high.

4.6 Are the generated models capable of extrapolating beyond training data and generalizing beyond training loading paths?

To answer this question, we use Chart d) of Figures 8, 9, and 10, which evaluate the models on loading scenarios that were not part of the training. For the synthetic rubber material, ground-truth stresses can be computed for arbitrary deformations, which allows evaluation on Treloar’s invariant plane Treloar (2005). The

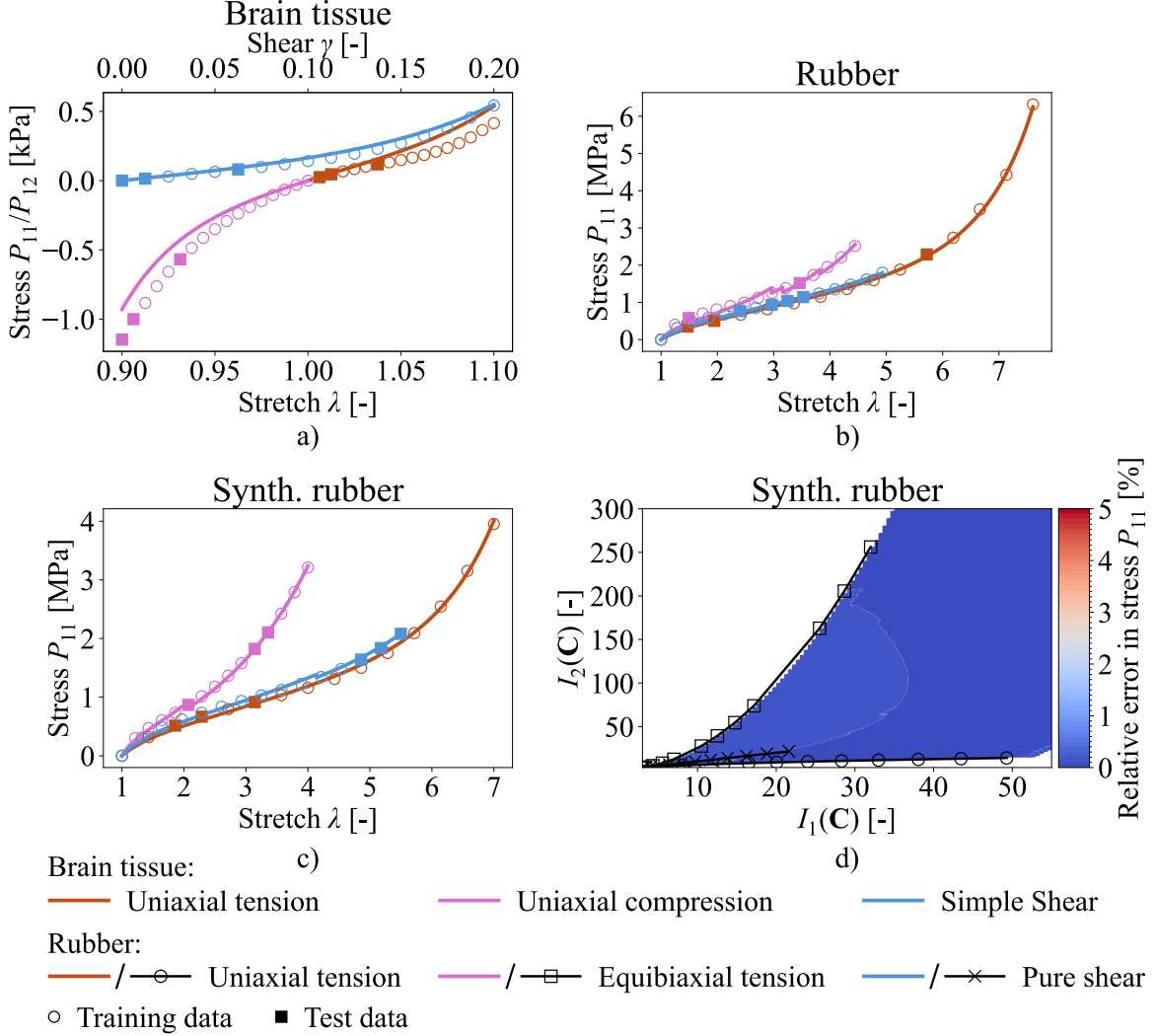


Figure 8: Predictions of models generated with Claude Opus 4.7. For each case we show the best-out-of-ten adherent model obtained with Inspector without tool access. Charts a)–c) show stress predictions on the training loading conditions, while Chart d) shows predictions on Treloar’s invariant plane Treloar (2005) for the synthetic rubber material, assessing generalization and extrapolation capabilities.

first and second invariants span the x- and y-axes; the plane ranges from uniaxial to equibiaxial tension, with pure shear at the angular midpoint (note that the two axes use different scales). Only these three highlighted loading paths are used during training, so every point in between or beyond corresponds to a loading state that the model has never seen. This setup lets us probe both generalization (to intermediate loading states) and extrapolation (beyond the training range).

Figure 10 d) shows that the baseline model generalizes well to unseen loading conditions but has small difficulties when extrapolating significantly beyond the training data. The relative errors remain well under 5%, but they are clearly noticeable. The models generated by our pipeline, in contrast, generalize and extrapolate remarkably well: within the range evaluated here, their accuracy does not drop noticeably when moving away from the training loading paths or beyond the training range.

4.7 Are the generated models comparable in parameter counts to the baselines?

A recurring concern when comparing the accuracy of different constitutive models is that parameter counts should be matched. It is worth emphasizing that the goal of this work is not to match or beat the accuracy

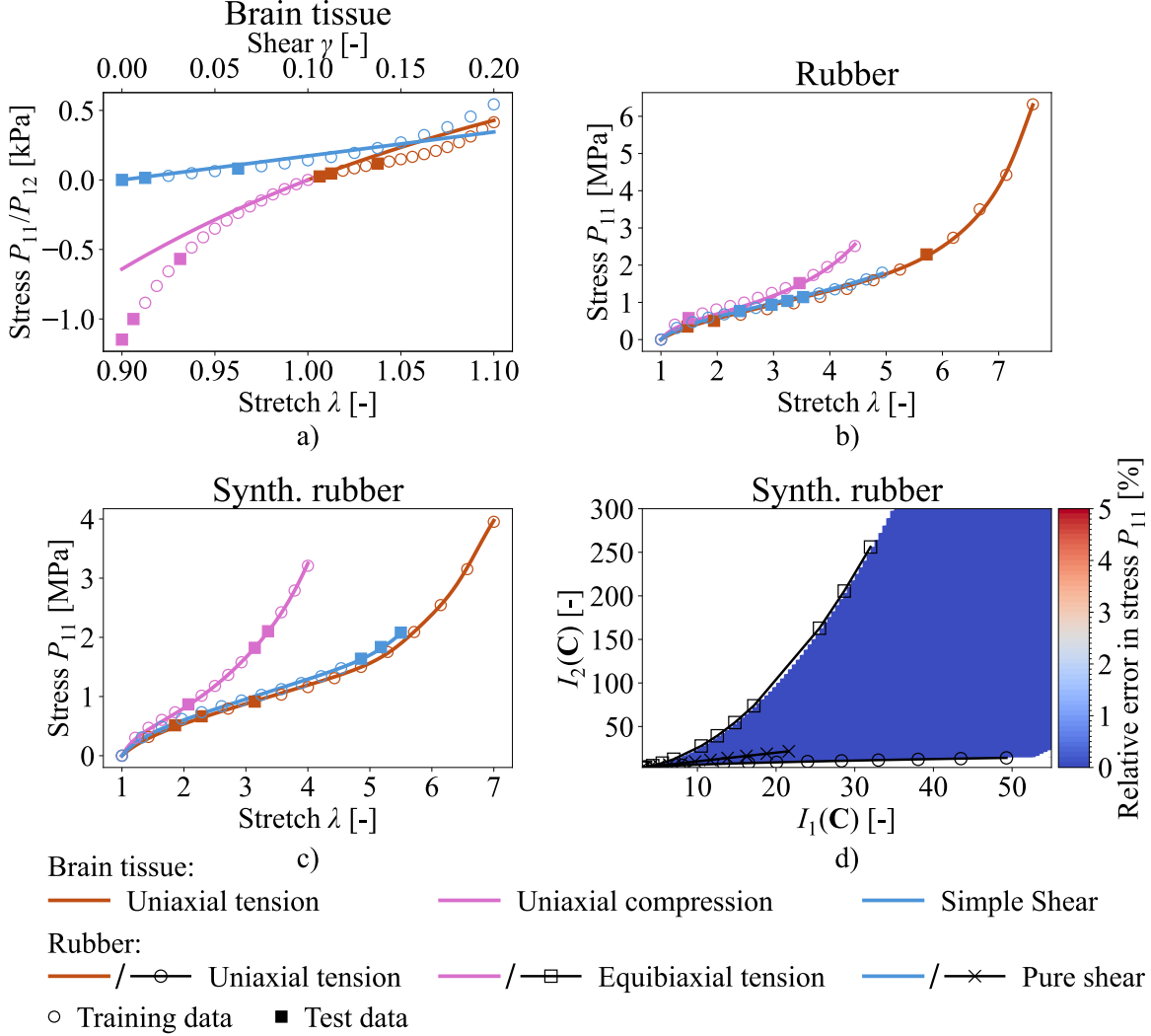


Figure 9: Predictions of models generated with Kimi K2.5. For each case we show the best-out-of-ten adherent model obtained with Inspector without tool access. Charts a)–c) show stress predictions on the training loading conditions, while Chart d) shows predictions on Treloar’s invariant plane Treloar (2005) for the synthetic rubber material, assessing generalization and extrapolation capabilities.

of human-expert-designed models, but rather to understand what a multi-agent system powered by today’s frontier LLMs can do in terms of producing both accurate and physically meaningful constitutive models. Still, a brief look at network sizes helps put the reported accuracies into perspective.

In the GenCANN work Tacke et al. (2025a), the LLM was, in one variant of the setup, explicitly instructed to reproduce the baseline’s network size so as to enable a head-to-head comparison. In the realistic setting targeted here — where either a single LLM or, as in our case, a specialized multi-agent system designs a constitutive model for a new material for which no established model exists — such a head-to-head constraint is not meaningful. The useful question is rather whether restricting the network size damages the LLM’s ability to design a working model. The answer given in Tacke et al. (2025a) is clear: the accuracy loss from restricting the network size is small, and the LLMs are robust in this regard. Since this question has already been settled, we did not repeat the corresponding experiments here.

For transparency, however, we report the network sizes of the models whose predictions are shown in Figures 8–10 in Table 1. Two observations stand out. First, the Creator-Inspector pair powered by Claude Opus, entirely by its own design choices, tends to produce networks of a size comparable to the baselines. Small networks are always welcome in terms of runtime, so this is a favorable characteristic. Second, the Kimi-powered pair

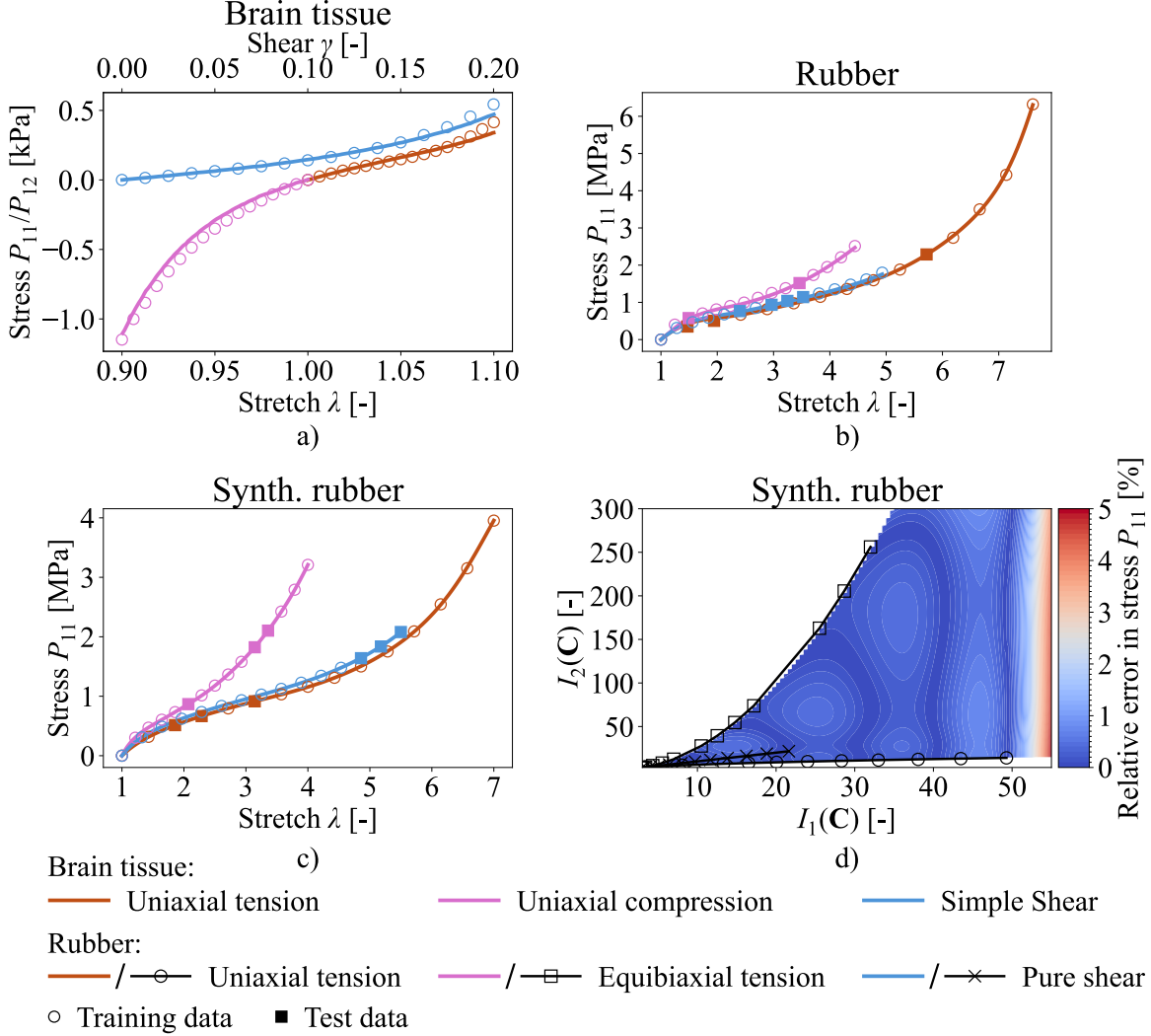


Figure 10: Predictions of models designed by human experts that serve as baselines. For brain tissue, we take as baseline the most accurate CANN reported across several optimization studies on this dataset Linka et al. (2023a); McCulloch et al. (2024); Pierre et al. (2023a); Budday et al. (2019), namely the one from Pierre et al. (2023a). For both rubber datasets — the experimental and the synthetic one — we use the most accurate CANN from the original publication Linka et al. (2021). Charts a)–c) show stress predictions on the training loading conditions, while Chart d) shows predictions on Treloar’s invariant plane Treloar (2005) for the synthetic rubber material, assessing generalization and extrapolation capabilities.

tends to produce larger networks than the baseline. This is consistent with the observations in Tacke et al. (2025a), where unconstrained LLMs showed a tendency toward oversized architectures.

The decisive criterion, however, is not the absolute network size but whether the generated models overfit the small training datasets. As the generalization and extrapolation results in Section 4.6 demonstrate, they clearly do not. The network sizes chosen by the agents are therefore appropriate for the task, and the experiments in (Tacke et al., 2025a) make us confident that comparable accuracy could be reached with smaller networks if desired.

4.8 How reliably do the generated models reach the presented accuracy?

Since the predictions shown so far correspond to the best out of ten runs, a natural follow-up question is how reliably the Creator–Inspector pair produces models of comparable accuracy. Figure 11 answers this. Each bar

Table 1: Architectural specifications of constitutive artificial neural networks (CANNs) used as baseline and generated by Creator–Inspector (Claude Opus 4.7 and Kimi K2.5, no tool access), with predictions shown in Figures 8 - 10.

Material	Model	Neurons per hidden layer
Brain tissue	Opus-generated	12, 12
	Kimi-generated	128, 128, 128, 128
	Baseline	100
Rubber	Opus-generated	20, 20
	Kimi-generated	10, 10
	Baseline	16, 16
Synth. rubber	Opus-generated	24, 12
	Kimi-generated	256, 256
	Baseline	16, 16

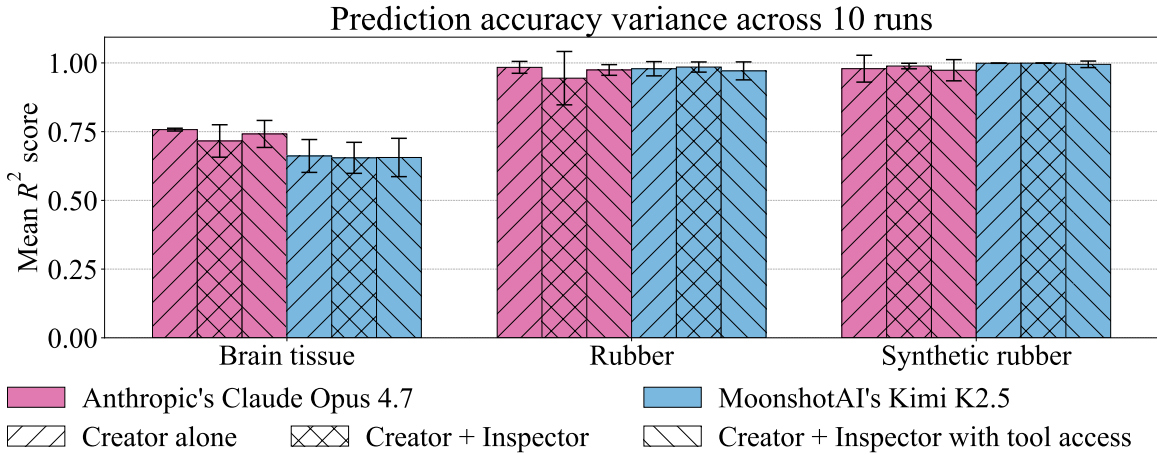


Figure 11: Reliability of the achieved accuracy across runs. Mean R^2 score of model performance, averaged over the three loading conditions of each dataset and over ten independent runs and showing also standard deviation

shows the mean R^2 score, averaged over the three loading conditions of the respective dataset and over ten runs, with the standard deviation indicated by a line on top. We report this for every configuration: Creator alone, Creator with Inspector, and Creator with tool-enabled Inspector, each for Claude Opus 4.7 and Kimi K2.5, on every dataset.

Across all configurations and datasets, the variance between runs is small. On both rubber datasets it is close to zero, since virtually all runs reach near-perfect accuracy. The one visible exception is the Opus-powered Creator–Inspector pair on the experimental rubber dataset, where the mean drops slightly and the standard deviation grows accordingly; this is caused by a single outlier run with an R^2 score of 0.66 averaged over the three loading conditions. Since the pipeline is partly stochastic, such individual cases can occur and account for the small dip. On the brain tissue data the variance is somewhat larger — more pronounced for the Kimi-powered agents than for the Opus-powered ones — but still modest. Overall, the accuracies shown in Figures 8 - 10 are not the results of lucky individual runs but are representative of what the pipeline delivers in a typical execution.

4.9 How effective is the iterative refinement of the models?

Since our pipeline includes a refinement loop, we should check whether that loop actually has an effect on the generated models. To this end, we pool the six configurations analyzed previously — Creator alone, Creator with Inspector, and Creator with tool-enabled Inspector, each for Opus and Kimi — into a single group

of experiments, but separate the results by refinement round and by dataset. The corresponding accuracies are shown in Figure 12. For readability, we clip negative average R^2 scores to zero, as a small number of extreme outliers would otherwise dominate the plot. This clipping was not necessary in Figure 11, because the refinement process itself filtered out these drastic errors — already a first indication that the loop gives the LLMs room to experiment and to discard clearly broken models before the final export.

The variance across runs in Figure 12 is visibly larger than in Figure 11. For the two rubber datasets, no clear trend across the three refinement rounds is discernible: the standard deviations are simply too large to support any significant statement. For the brain tissue data, the variance is much smaller, and a small improvement after the first refinement is visible, while the second refinement does not yield any further improvement.

Taken together, these patterns suggest that the refinement rounds serve primarily as an exploration opportunity for the LLMs. On the rubber datasets, where the agents reliably produce at least one highly accurate model early on — as reflected in the tiny variance of the final results in Figure 11 — the remaining rounds are evidently used to try out more unconventional ideas, which inflates the per-round variance. On the brain tissue data, where obtaining an accurate model is harder, the agents also explore but stay more focused: the variance is larger than in Figure 11 but visibly smaller than for rubber in Figure 12, and the small improvement after the first refinement shows that incremental progress does occur where it is most needed. In both cases the refinement loop is used productively, just in different ways depending on how difficult the modeling task is.

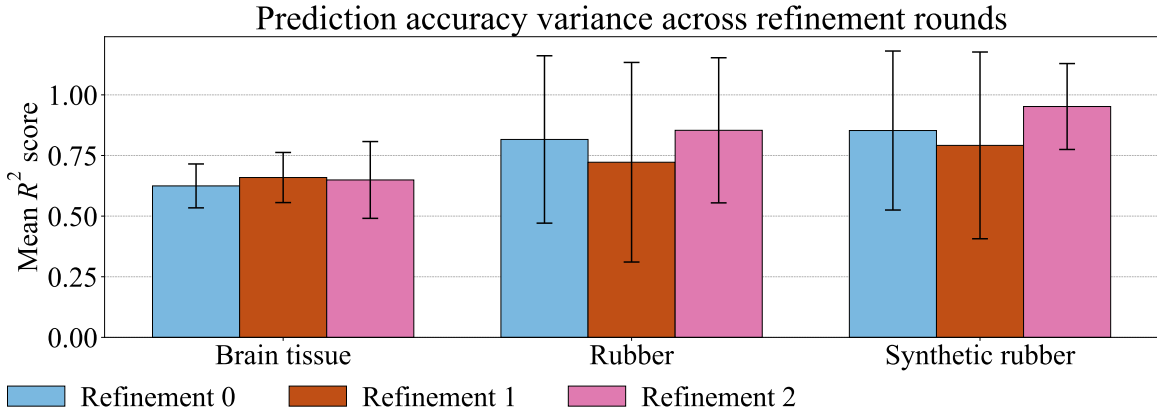


Figure 12: Effect of the iterative refinement loop. Mean R^2 score (including standard deviation) of the generated model per refinement round, pooled across all six configurations (Creator alone, Creator with Inspector, and Creator with tool-enabled Inspector, each powered by Claude Opus 4.7 and Kimi K2.5) and shown separately for each dataset. Negative average R^2 scores were clipped to zero to keep extreme outliers from dominating the plot.

5 Discussion and Conclusions

5.1 Effectiveness of the multi-agent setup: two agents are better than one

The central methodological contribution of this work is the Creator–Inspector paradigm. Our analysis of the state transitions between Creator and Inspector shows that the two agents communicate effectively. For the Opus-powered pair, whenever the Inspector raises a new violation, the Creator resolves it in the very next iteration in 93% of cases, and violations persist in only 3%. For the Kimi-powered pair, still 69% of new violations are resolved directly and only 21% persist. In other words, when the Inspector sees the need to intervene, its feedback is actionable rather than merely critical. This is the behavior one would hope for from a generator–critic pair, and it mirrors the empirical case that has been made for multi-agent LLM systems more broadly, where a clean separation between proposing and inspecting roles consistently outperforms monolithic chain-of-thought pipelines (Du et al., 2024; Gridach et al., 2025; Chen et al., 2024b; Liu et al., 2024a). For constitutive modeling, however, no prior single-agent system — SGA (Ma et al., 2024), CSGA (Tacke et al., 2025b), or GenCANN (Tacke et al., 2025a) — had closed this gap.

The effect of this collaboration on the final output is striking. Introducing the Inspector on top of a Creator powered by Claude Opus 4.7 raises the share of proposed models that fulfill all nine physical constraints from

91% to 100%. While we cannot guarantee that our framework is free of mistakes in general, reaching 100% reliability on our test cases for the specific problem of generating constitutive models for hyperelastic, isotropic, incompressible materials marks a clear milestone for the trustworthy use of LLMs in constitutive modeling. For Kimi K2.5, one of the most capable open-source LLMs currently available, the absolute reliability is lower. The characteristics of any specific LLM are, however, of limited interest to us, as frontier models evolve too quickly to make claims tied to a particular release. What matters is that the Inspector lifts reliability also for Kimi, from 37% to 56%. Since the same effect emerges in two LLMs that otherwise behave very differently, the improvement can be attributed to the paradigm itself rather than to any model-specific behavior.

Reliability is only one side of the coin; accuracy is the other. Figures 8–11 show that the generated models are not only physically valid but also highly accurate, with the Opus-powered models reaching performance nearly on par with the models designed by human experts. Equally important, both the Opus- and Kimi-powered models generalize and extrapolate remarkably well beyond the training loading conditions, as evidenced by their predictions on Treloar’s invariant plane. Together, this combination of physical validity, high accuracy, and strong extrapolation makes the generated models directly usable across a wide range of applications.

Beyond these concrete results, it is worth stressing that the paradigm itself is deliberately technique-agnostic. Our use of CANNs is a concrete demonstration vehicle, chosen because CANNs are well established and offer a favorable trade-off between flexibility and physical structure, but the same pipeline could be instantiated around PANNs, CKANs, or even purely symbolic laws without conceptual changes — only the prompts given to the Creator and Inspector would need to be adapted. In this sense, the Creator–Inspector paradigm closes the gap left open by GenCANN — which produced accurate but physically unchecked models — and provides a general template for physics-aware LLM-driven constitutive modeling.

5.2 Autonomy of the agents: calibrated to match what today’s LLMs can reliably deliver

Agentic LLM-powered systems can be more or less autonomous, and it is worth being explicit about where our pipeline sits on this spectrum. Two design choices constrain the autonomy of the agents: the alternation between Creator and Inspector is predefined rather than negotiated by the agents themselves, and the numerical validation tools available to the Inspector are hard-coded rather than implemented on the fly. Within these constraints, however, both agents retain meaningful freedom. The Creator autonomously decides on the architecture, the invariant-based building blocks, the activation functions, and the training procedure of each proposed CANN. The Inspector decides how many creation updates are triggered - implicitly, through its verdicts - and, in the tool-enabled variant, which tools to invoke and how many. On balance, the Creator is best described as constrained but expressive, and the Inspector as semi-autonomous.

Autonomy, however, is not a value in itself; it is only useful insofar as it translates into a practical advantage. The two constraints we imposed are motivated precisely by this consideration. Fixing the Creator-Inspector alternation reflects the fact that, at the current level of LLM reliability, it is simply not safe to let the Creator decide whether inspection is needed: our results in Section 4.2 show that even a frontier model such as Claude Opus 4.7 produces physically invalid models in roughly one out of ten proposals when left unchecked. If, at some point in the future, Creators become reliable enough to estimate their own constraint adherence, one could naturally relax this constraint and let the Creator call the Inspector on demand. A similar argument applies to the hard-coded tools: letting the Inspector implement numerical validation routines on the fly would currently add a new and uncontrolled source of errors, but once LLMs routinely produce correct and trustworthy validation code, the external tools would largely lose their purpose anyway. The balance between autonomy and reliability that we chose here is therefore not meant as a universal prescription, but as the one that, in our view, best matches the current capabilities of the underlying LLMs.

5.3 Practical usability: scales automatically with LLM and agentic progress

What does this mean for someone who wants to use the pipeline in practice? Our experiments can be read, in part, as a capability study: they probe how far today’s frontier LLMs can be pushed, within a carefully designed agentic setup, toward the task of generating physically admissible constitutive models. The answer is encouraging. For hyperelastic, isotropic, incompressible materials, the Creator–Inspector pair powered by Claude Opus 4.7 is ready to produce constitutive models that respect the relevant physical constraints, are highly accurate, and generalize to unseen loading conditions as well as extrapolate beyond the training data. In the future, the same approach can be tested on a broader range of material classes. A further practical strength

is the form in which the models are returned: plain Python files that can be inspected, modified, and reused directly. And should the user wish to modify the model or any aspect of it, a plain-text prompt is enough to request the change.

Two developments are likely to push this horizon further still. The first concerns the agentic design itself. Our pipeline is, to the best of our knowledge, the first multi-agent system for constitutive modeling, and there is substantial room for improvement through additional agents, longer refinement loops, and more carefully engineered prompts. The impact of such refinements should not be underestimated: on the widely used HumanEval coding benchmark, GPT-3.5 reaches 48.1% in a zero-shot setting and GPT-4 reaches 67.0%, but wrapping GPT-3.5 in an iterative agentic workflow pushes it to 95.1% (Ng, 2024) — an improvement that dwarfs the underlying model upgrade. In other words, the agentic scaffolding around an LLM can matter more than the LLM itself, and the design space we have begun to explore here is still largely open.

The second development concerns the LLM backbones. As they continue to improve, we expect both the accuracy of the generated models and their adherence to physical constraints to rise with them, and we expect the Creator and Inspector to become trustworthy enough to operate with increasing autonomy. Switching to a more capable backbone is a one-line change in our codebase, so the pipeline is well positioned to ride this curve rather than fight it. In the spirit of Sutton’s bitter lesson — that general methods which scale with computation eventually outperform approaches relying on hand-engineered domain knowledge (Sutton, 2019) — the Creator–Inspector paradigm is deliberately built to benefit from stronger LLMs rather than to depend on any particular one.

Declarations

Software and data availability

The human brain tissue dataset and the corresponding baseline CANN implementation are available at <https://github.com/LivingMatterLab/CANN>. The rubber datasets and their corresponding baseline CANN implementation can be found at <https://github.com/ConstitutiveANN/CANN>. The code for the approach presented in this work, together with all artifacts reported here, including the full set of prompts and the complete conversation logs between the Creator and Inspector agents, is available at <https://github.com/AgenticModeling/MultiAgentConstitutiveModeling26>.

Author contributions

Marius Tacke conceived and designed the study, developed the methodology and software, curated the data, and carried out the investigation, validation, and formal analysis. He also wrote the original draft. Matthias Busch contributed to the formal analysis and prepared visualizations. Kian Abdolazizi contributed to the methodology and validation. Jonas Eichinger contributed to the methodology. Kevin Linka contributed to the conceptualization of the study. Roland Aydin contributed to the conceptualization and visualization and supervised the work. Christian Cyron provided resources, contributed to the visualization, supervised the work, and administered the project. All authors reviewed and edited the manuscript.

Funding

This research did not receive funding.

Competing interests

The authors declare no competing interests.

Ethical approval

No animals or human subjects were used in this study. All data are taken from the literature.

References

- Kian P Abdolazizi, Kevin Linka, and Christian J Cyron. Viscoelastic constitutive artificial neural networks (vcanns)—a framework for data-driven anisotropic nonlinear finite viscoelasticity. *Journal of computational physics*, 499:112704, 2024. doi: <https://doi.org/10.1016/j.jcp.2023.112704>.
- Kian P Abdolazizi, Roland C Aydin, Christian J Cyron, and Kevin Linka. Constitutive kolmogorov–arnold networks (ckans): Combining accuracy and interpretability in data-driven material modeling. *Journal of the Mechanics and Physics of Solids*, pp. 106212, 2025. doi: <https://doi.org/10.1016/j.jmps.2025.106212>.
- Rasul Abdusalamov, Markus Hillgärtner, and Mikhail Itskov. Automatic generation of interpretable hyperelastic material models by symbolic regression. *International Journal for Numerical Methods in Engineering*, 124(9):2093–2104, 2023. doi: <https://doi.org/10.1002/nme.7203>.
- Faisal As’ad, Philip Avery, and Charbel Farhat. A mechanics-informed artificial neural network approach in data-driven constitutive modeling. *International Journal for Numerical Methods in Engineering*, 123(12): 2738–2759, 2022. doi: <https://doi.org/10.1002/nme.6957>.
- Adib Bazgir, Yuwen Zhang, et al. Matagent: a human-in-the-loop multi-agent llm framework for accelerating the material science discovery cycle. In *AI for Accelerated Materials Design-ICLR 2025*, 2025.
- Daniil A Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023.
- Ludwig Boltzmann. Zur theorie der elastischen nachwirkung. *Sitzungsber. kais. Akad. Wiss. Wien, Math.-Nat. Classe.*, 70:275, 1874.
- Onur Boyar, Lisa Hamada, Seiji Takeda, et al. Llm-fusion: A novel multimodal fusion model for accelerated material discovery. In *AAAI Conference on Artificial Intelligence*, 2025.
- Steven L Brunton, Joshua L Proctor, and J Nathan Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the national academy of sciences*, 113(15): 3932–3937, 2016.
- Silvia Budday, Gerhard Sommer, Christoph Birkel, Christian Langkammer, Johannes Haybaeck, Julius Kohnert, Melanie Bauer, Friedrich Paulsen, Paul Steinmann, Ellen Kuhl, et al. Mechanical characterization of human brain tissue. *Acta biomaterialia*, 48:319–340, 2017a. doi: <https://doi.org/10.1016/j.actbio.2016.10.036>.
- Silvia Budday, Gerhard Sommer, Johannes Haybaeck, Paul Steinmann, Gerhard A Holzapfel, and Ellen Kuhl. Rheological characterization of human brain tissue. *Acta biomaterialia*, 60:315–329, 2017b. doi: <https://doi.org/10.1016/j.actbio.2017.06.024>.
- Silvia Budday, Timothy C Ovaert, Gerhard A Holzapfel, Paul Steinmann, and Ellen Kuhl. Fifty shades of brain: a review on the mechanical testing and modeling of brain tissue. *Archives of Computational Methods in Engineering*, 2019. doi: <https://doi.org/10.1007/s11831-019-09352-w>.
- Pietro Carrara, Laura De Lorenzis, Laurent Stainier, and Michael Ortiz. Data-driven fracture mechanics. *Computer Methods in Applied Mechanics and Engineering*, 372:113390, 2020. doi: <https://doi.org/10.1016/j.cma.2020.113390>.
- Akshat Chaudhari, Janghoon Ock, and Amir Barati Farimani. Modular large language model agents for multi-task computational materials science. *Communications Materials*, 2026.
- Angelica Chen, Samuel Don Stanton, Robert G Alberstein, Andrew Martin Watkins, Richard Bonneau, Vladimir Gligorijevic, Kyunghyun Cho, and Nathan C. Frey. LLMs are highly-constrained biophysical sequence optimizers. In *NeurIPS 2024 Workshop on AI for New Drug Modalities*, 2024a. URL <https://openreview.net/forum?id=SzfRVq8X07>.
- Guangyao Chen, Siwei Dong, Yu Shu, Ge Zhang, Jaward Sesay, Börje Karlsson, Jie Fu, and Yemin Shi. Autoagents: a framework for automatic agent generation. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, pp. 22–30, 2024b. doi: <https://doi.org/10.24963/ijcai.2024/3>.
- Hüsni Dal, Funda Aksu Denli, Alp Kağan Acan, and Michael Kaliske. Data-driven hyperelasticity, part i: A canonical isotropic formulation for rubberlike materials. *Journal of the Mechanics and Physics of Solids*, 179:105381, 2023. doi: <https://doi.org/10.1016/j.jmps.2023.105381>.
- Franz Dammaß, Karl A Kalina, and Markus Kästner. Neural networks meet phase-field: A hybrid fracture model. *Computer Methods in Applied Mechanics and Engineering*, 440:117937, 2025.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first international conference on machine learning*, 2024.

- Alexander Edmund Ehret and Alberto Stracuzzi. Variations on ogden’s model: close and distant relatives. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 380 (2234), 2022.
- Moritz Flaschel, Siddhant Kumar, and Laura De Lorenzis. Unsupervised discovery of interpretable hyperelastic constitutive laws. *Computer Methods in Applied Mechanics and Engineering*, 381:113852, 2021. doi: <https://doi.org/10.1016/j.cma.2021.113852>.
- Moritz Flaschel, Siddhant Kumar, and Laura De Lorenzis. Discovering plasticity models without stress data. *npj Computational Materials*, 8(1):91, 2022. doi: <https://doi.org/10.1038/s41524-022-00752-4>.
- Konrad Friedrichs, Franz Dammaß, Karl A Kalina, and Markus Kästner. Precise, efficient and flexible modeling of crystallizing elastomers based on physics-augmented neural networks. *Computer Methods in Applied Mechanics and Engineering*, 455:118852, 2026.
- Frederick N Fritsch and Ralph E Carlson. Monotone piecewise cubic interpolation. *SIAM Journal on Numerical Analysis*, 17(2):238–246, 1980.
- Jan N Fuhg, Govinda Anantha_Padmanabha, Nikolaos Bouklas, Bahador Bahmani, WaiChing Sun, Nikolaos N Vlassis, Moritz Flaschel, Pietro Carrara, and Laura De_Lorenzis. A review on data-driven constitutive laws for solids. *Archives of Computational Methods in Engineering*, 2024. doi: <https://doi.org/10.1007/s11831-024-10196-2>.
- Gian-Luca Geuken, Patrick Kurzeja, David Wiedemann, Martin Zlatić, Marko Čanadija, and Jörn Mosler. Modeling isotropic polyconvex hyperelasticity by neural networks—sufficient and necessary criteria for compressible and incompressible materials. *arXiv preprint arXiv:2603.27351*, 2026.
- Jamshid Ghaboussi, JH Garrett Jr, and Xiping Wu. Knowledge-based modeling of material behavior with neural networks. *Journal of engineering mechanics*, 117(1):132–153, 1991. doi: [https://doi.org/10.1061/\(ASCE\)0733-9399\(1991\)117:1\(132\)](https://doi.org/10.1061/(ASCE)0733-9399(1991)117:1(132)).
- Alireza Ghafarollahi and Markus J Buehler. Protagonists: protein discovery via large language model multi-agent collaborations combining physics and machine learning. *Digital Discovery*, 3(7):1389–1409, 2024.
- Alireza Ghafarollahi and Markus J Buehler. Sciagents: automating scientific discovery through bioinspired multi-agent intelligent graph reasoning. *Advanced Materials*, 37(22):2413523, 2025.
- David González, Francisco Chinesta, and Elías Cueto. Thermodynamically consistent data-driven computational mechanics. *Continuum Mechanics and Thermodynamics*, 31(1):239–253, 2019.
- Mourad Gridach, Jay Nanavati, Christina Mack, Khaldoun Zine El Abidine, and Lenon Mendes. Agentic AI for scientific discovery: A survey of progress, challenges, and future directions. In *Towards Agentic AI for Science: Hypothesis Generation, Comprehension, Quantification, and Validation*, 2025.
- Yuchuan Gu, Lusheng Wang, Jun Ding, Yanhong Peng, Changgeng Li, and Shaojie Gu. Large language model-enabled constitutive modeling for rate-dependent plasticity and automatic umat subroutine generation. *Computers, Materials, & Continua*, 87(2), 2026.
- Hao Hao, Xiaoqun Zhang, and Aimin Zhou. Large language models as surrogate models in evolutionary algorithms: A preliminary study. *Swarm and Evolutionary Computation*, 91:101741, 2024. doi: <https://doi.org/10.1016/j.swevo.2024.101741>.
- Zhongkai Hao, Songming Liu, Yichi Zhang, Chengyang Ying, Yao Feng, Hang Su, and Jun Zhu. Physics-informed machine learning: A survey on problems, methods and applications. *arXiv preprint arXiv:2211.08064*, 2022. doi: <https://doi.org/10.48550/arXiv.2211.08064>.
- Youssef MA Hashash, Sungmoon Jung, and Jamshid Ghaboussi. Numerical implementation of a neural network based material model in finite element analysis. *International Journal for numerical methods in engineering*, 59(7):989–1005, 2004. doi: <https://doi.org/10.1002/nme.905>.
- Mathis Heyer, Jiyizhe Zhang, Naoto Sugisawa, Jan-Frederic Laub, and Alexei Lapkin. Automated generation of mechanistic models for chemical process digital twins using reinforcement learning-part i: Conceptual framework and equation generation. *ChemRxiv preprint*, 2025. doi: <https://doi.org/10.26434/chemrxiv-2025-r70bs-v3>.
- Hagen Holthusen, Lukas Lamm, Tim Brepols, Stefanie Reese, and Ellen Kuhl. Theory and implementation of inelastic constitutive artificial neural networks. *Computer Methods in Applied Mechanics and Engineering*, 428:117063, 2024.
- Hagen Holthusen, Tim Brepols, Kevin Linka, and Ellen Kuhl. Automated model discovery for tensional homeostasis: Constitutive machine learning in growth and remodeling. *Computers in biology and medicine*, 186:109691, 2025a.

- Hagen Holthausen, Kevin Linka, Ellen Kuhl, and Tim Brepols. A generalized dual potential for inelastic constitutive artificial neural networks: A jax implementation at finite strains. *Journal of the Mechanics and Physics of Solids*, pp. 106337, 2025b.
- Gerhard A Holzapfel. *NONLINEAR SOLID MECHANICS. A Continuum Approach for Engineering*. John Wiley & Sons, Chichester, second print edition, 2001.
- Robert Hooke. *De Potentia Restitutiva, or of Spring: Explaining the Power of Springing Bodies*. John Martyn, London, 1678. Introduced the relation later summarized as Hooke’s law.
- Bo Hu, Siyu Liu, Beilin YE, David J. Srolovitz, and Tongqi Wen. A multi-agent framework for physical laws discovery. In *AI4X Accelerate Conference 2026*, 2026.
- Jiao Huang, Qianli Xing, Jinglong Ji, and Bo Yang. Code-generated graph representations using multiple LLM agents for material properties prediction. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=lvvMwGUam6>.
- Chenyi Ji, Kian P Abdolazizi, Hagen Holthausen, Christian J Cyron, and Kevin Linka. Inelastic constitutive kolmogorov-arnold networks: A generalized framework for automated discovery of interpretable inelastic material models. *arXiv preprint arXiv:2602.17750*, 2026.
- Shuyi Jia, Chao Zhang, and Victor Fung. Lmatdesign: Autonomous materials discovery with large language models. *arXiv preprint arXiv:2406.13163*, 2024.
- Akshay Joshi, Prakash Thakolkaran, Yiwen Zheng, Maxime Escande, Moritz Flaschel, Laura De Lorenzis, and Siddhant Kumar. Bayesian-euclid: Discovering hyperelastic material laws with uncertainties. *Computer Methods in Applied Mechanics and Engineering*, 398:115225, 2022. doi: <https://doi.org/10.1016/j.cma.2022.115225>.
- Karl A Kalina, Jörg Brummund, WaiChing Sun, and Markus Kästner. Neural networks meet anisotropic hyperelasticity: A framework based on generalized structure tensors and isotropic tensor functions. *Computer Methods in Applied Mechanics and Engineering*, 437:117725, 2025.
- Trenton Kirchdoerfer and Michael Ortiz. Data-driven computational mechanics. *Computer Methods in Applied Mechanics and Engineering*, 304:81–101, 2016. doi: <https://doi.org/10.1016/j.cma.2016.02.001>.
- Dominik K Klein, Mauricio Fernández, Robert J Martin, Patrizio Neff, and Oliver Weeger. Polyconvex anisotropic hyperelasticity with neural networks. *Journal of the Mechanics and Physics of Solids*, 159: 104703, 2022.
- A. N. Kolmogorov. On the representation of continuous functions of several variables by superpositions of continuous functions of a smaller number of variables. *Dokl. Akad. Nauk SSSR*, 108:179–182, 1956.
- John R Koza. Genetic programming: On the programming of computers by means of natural selection (complex adaptive systems). *A Bradford Book*, 1:18, 1993.
- Marcos Latorre and Francisco Javier Montáns. Extension of the sussman–bathe spline-based hyperelastic model to incompressible transversely isotropic materials. *Computers & Structures*, 122:13–26, 2013. doi: <https://doi.org/10.1016/j.compstruc.2013.01.018>.
- Marcos Latorre and Francisco Javier Montáns. What-you-prescribe-is-what-you-get orthotropic hyperelasticity. *Computational Mechanics*, 53(6):1279–1298, 2014.
- Sanghee Lee and Klaus-Jürgen Bathe. A constitutive neural network for incompressible hyperelastic materials. *Machine Learning for Computational Science and Engineering*, 1(2):31, 2025.
- Shanda Li, Tanya Marwah, Junhong Shen, Weiwei Sun, Andrej Risteski, Yiming Yang, and Ameet Talwalkar. Codepde: An inference framework for llm-driven pde solver generation. *arXiv preprint arXiv:2505.08783*, 2025. doi: <https://doi.org/10.48550/arXiv.2505.08783>.
- Zuhong Lin, Daoyuan Ren, Kai Ran, Jing Sun, Songlin Yu, Xuefeng Bai, Xiaotian Huang, Haiyang He, Pengxu Pan, Ying Fang, et al. Reshaping mofs text mining with a dynamic multi-agents framework of large language model. *arXiv preprint arXiv:2504.18880*, 2025.
- Lennart Linden, Dominik K Klein, Karl A Kalina, Jörg Brummund, Oliver Weeger, and Markus Kästner. Neural networks meet hyperelasticity: A guide to enforcing physics. *Journal of the Mechanics and Physics of Solids*, 179:105363, 2023. doi: <https://doi.org/10.1016/j.jmps.2023.105363>.
- Kevin Linka and Ellen Kuhl. A new family of constitutive artificial neural networks towards automated model discovery. *Computer Methods in Applied Mechanics and Engineering*, 403:115731, 2023.

- Kevin Linka, Markus Hillgärtner, Kian P Abdolazizi, Roland C Aydin, Mikhail Itskov, and Christian J Cyron. Constitutive artificial neural networks: A fast and general approach to predictive data-driven constitutive modeling by deep learning. *Journal of Computational Physics*, 429:110010, 2021. doi: <https://doi.org/10.1016/j.jcp.2020.110010>.
- Kevin Linka, Sarah R St Pierre, and Ellen Kuhl. Automated model discovery for human brain using constitutive artificial neural networks. *Acta Biomaterialia*, 160:134–151, 2023a. doi: <https://doi.org/10.1016/j.actbio.2023.01.055>.
- Kevin Linka, Adrian Buganza Tepole, Gerhard A Holzapfel, and Ellen Kuhl. Automated model discovery for skin: Discovering the best model, data, and experiment. *Computer Methods in Applied Mechanics and Engineering*, 410:116007, 2023b. doi: <https://doi.org/10.1016/j.cma.2023.116007>.
- Quanliang Liu, Maciej P Polak, So Yeon Kim, MD Al Amin Shuvo, Hrishikesh Shridhar Deodhar, Jeongsoo Han, Dane Morgan, and Hyunseok Oh. Beyond designer’s knowledge: Generating materials design hypotheses via a large language model. *Acta Materialia*, 297:121307, 2025.
- Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. A dynamic LLM-powered agent network for task-oriented agent collaboration. In *First Conference on Language Modeling*, 2024a. URL <https://openreview.net/forum?id=XII0Wp1XA9>.
- Ziming Liu, Pingchuan Ma, Yixuan Wang, Wojciech Matusik, and Max Tegmark. Kan 2.0: Kolmogorov-arnold networks meet science. *arXiv preprint arXiv:2408.10205*, 2024b. doi: <https://doi.org/10.48550/arXiv.2408.10205>.
- Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y Hou, and Max Tegmark. Kan: Kolmogorov-arnold networks. *arXiv preprint arXiv:2404.19756*, 2024c. doi: <https://doi.org/10.48550/arXiv.2404.19756>.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- Pingchuan Ma, Tsun-Hsuan Wang, Minghao Guo, Zhiqing Sun, Joshua B. Tenenbaum, Daniela Rus, Chuang Gan, and Wojciech Matusik. LLM and simulation as bilevel optimizers: A new paradigm to advance physical scientific discovery. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=hz8cFsdz7P>.
- James Clerk Maxwell. Iv. on the dynamical theory of gases. *Philosophical transactions of the Royal Society of London*, (157):49–88, 1867.
- Jeremy A McCulloch, Skyler R St. Pierre, Kevin Linka, and Ellen Kuhl. On sparse regression, lp-regularization, and automated model discovery. *International Journal for Numerical Methods in Engineering*, 125(14): e7481, 2024. doi: <https://doi.org/10.1002/nme.7481>.
- Melvin Mooney. A theory of large elastic deformation. *Journal of applied physics*, 11(9):582–592, 1940. doi: <https://doi.org/10.1063/1.1712836>.
- Patrizio Neff, Ionel-Dumitrel Ghiba, and Johannes Lankeit. The exponentiated hencky-logarithmic strain energy. part i: Constitutive issues and rank-one convexity. *Journal of Elasticity*, 121(2):143–234, 2015.
- Mohamed Nejjar, Luca Zacharias, Fabian Stiehle, and Ingo Weber. Llms for science: Usage for code generation and data analysis. *Journal of Software: Evolution and Process*, 37(1):e2723, 2025.
- Gayathri Nettem, M. Disha, Gilbert J. Aavish, Skanda Shreesha Prasad, and S. Natarajan. Agentflow: A context aware multi-agent framework for dynamic agent collaboration. In *Proceedings of the 17th International Conference on Agents and Artificial Intelligence*, pp. 687–693, 2025. doi: <https://doi.org/10.5220/0013375700003890>.
- Andrew Ng. Agentic design patterns part 1: Four ai agent strategies that improve gpt-4 and gpt-3.5 performance. *The Batch*, 20, 2024.
- Bo Ni and Markus J Buehler. Mechagents: Large language model multi-agent collaborations can solve mechanics problems, generate new data, and integrate knowledge. *Extreme Mechanics Letters*, 67:102131, 2024. doi: <https://doi.org/10.1016/j.eml.2024.102131>.
- Raymond William Ogden. Large deformation isotropic elasticity—on the correlation of theory and experiment for incompressible rubberlike solids. *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences*, 326(1567):565–584, 1972. doi: <http://doi.org/10.1098/rspa.1972.0026>.
- Sandeep Pandey, Ran Xu, Wenkang Wang, and Xu Chu. Openfoamgpt: a rag-augmented llm agent for openfoam-based computational fluid dynamics. *arXiv preprint arXiv:2501.06327*, 2025. doi: <https://doi.org/10.1063/5.0257555>.

- Dmitrii Pantiukhin, Boris Shapkin, Ivan Kuznetsov, Antonia Anna Jost, and Nikolay Koldunov. Accelerating earth science discovery via multi-agent llm systems. *Frontiers in Artificial Intelligence*, 8:1674927, 2025.
- Mathias Peirlinck, Kevin Linka, Juan A Hurtado, Gerhard A Holzapfel, and Ellen Kuhl. Democratizing biomedical simulation through automated model discovery and a universal material subroutine. *Computational mechanics*, 75(6):1703–1723, 2025.
- Sarah R St Pierre, Kevin Linka, and Ellen Kuhl. Principal-stretch-based constitutive neural networks autonomously discover a subclass of ogden models for human brain tissue. *Brain Multiphysics*, 4:100066, 2023a. doi: <https://doi.org/10.1016/j.brain.2023.100066>.
- Skyler R St Pierre, Divya Rajasekharan, Ethan C Darwin, Kevin Linka, Marc E Levenston, and Ellen Kuhl. Discovering the mechanics of artificial and real meat. *Computer Methods in Applied Mechanics and Engineering*, 415:116236, 2023b.
- Alejandro Polo-Molina, David Alfaya, and Jose Portela. Monokan: Certified monotonic kolmogorov-arnold network. *Neural Networks*, pp. 108278, 2025.
- L Prandtl. Spannungsverteilung in plastischen körpern proceedings of the 1st international congress on applied mechanics. *Delft*, 1925.
- ASME Reuss. Berücksichtigung der elastischen formänderung in der plastizitätstheorie. *ZAMM-Journal of Applied Mathematics and Mechanics/Zeitschrift für Angewandte Mathematik und Mechanik*, 10(3):266–274, 1930.
- Thiago Rios, Felix Lanfermann, and Stefan Menzel. Large language model-assisted surrogate modelling for engineering optimization. In *2024 IEEE Conference on Artificial Intelligence (CAI)*, pp. 796–803. IEEE, 2024. doi: <https://doi.org/10.1109/CAI59869.2024.00151>.
- Ronald S Rivlin. Large elastic deformations of isotropic materials iv. further developments of the general theory. *Philosophical transactions of the royal society of London. Series A, Mathematical and physical sciences*, 241(835):379–397, 1948a. doi: <http://doi.org/10.1098/rsta.1948.0024>.
- RSI Rivlin. Large elastic deformations of isotropic materials. i. fundamental concepts. *Philosophical Transactions of the Royal Society of London. Series A, Mathematical and Physical Sciences*, 240(822): 459–490, 1948b. doi: <http://doi.org/10.1098/rsta.1948.0002>.
- Samiha Sharlin and Tyler R Josephson. In context learning and reasoning for symbolic regression with large language models. *arXiv preprint arXiv:2410.17448*, 2024.
- Tongyu Shi, Yutang Li, Zhanlong Wang, Wenhe Xu, Guolai Jiang, Dawei Dai, Jie Zhou, Hao Huang, Rui He, Seeram Ramakrishna, et al. Knowledge-driven autonomous materials research via collaborative multi-agent and robotic system. *Matter*, 2026.
- Zhuofan Shi, Chunxiao Xin, Tong Huo, Yuntao Jiang, Bowen Wu, Xingyue Chen, Wei Qin, Xinjian Ma, Gang Huang, Zhenyu Wang, et al. A fine-tuned large language model based molecular dynamics agent for code generation to obtain material thermodynamic parameters. *Scientific Reports*, 15(1):10295, 2025. doi: <https://doi.org/10.1038/s41598-025-92337-6>.
- Parshin Shojaee, Kazem Meidani, Shashank Gupta, Amir Barati Farimani, and Chandan K Reddy. Llm-sr: Scientific equation discovery via programming with large language models. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Haoyang Su, Renqi Chen, Shixiang Tang, Zhenfei Yin, Xinzhe Zheng, Jinzhe Li, Biqing Qi, Qi Wu, Hui Li, Wanli Ouyang, et al. Many heads are better than one: Improved scientific idea generation by a llm-based multi-agent system. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 28201–28240, 2025.
- Theodore Sussman and Klaus-Jürgen Bathe. A model of incompressible isotropic hyperelastic material behavior using spline interpolations of tension–compression test data. *Communications in numerical methods in engineering*, 25(1):53–63, 2009. doi: <https://doi.org/10.1002/cnm.1105>.
- Rich Sutton. The bitter lesson, 2019. URL <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>. Blog post.
- Kyle Swanson, Wesley Wu, Nash L Bulaong, John E Pak, and James Zou. The virtual lab of ai agents designs new sars-cov-2 nanobodies. *Nature*, 646(8085):716–723, 2025.
- Vahidullah Taç, Francisco Sahli Costabal, and Adrian B Tepole. Data-driven tissue mechanics with polyconvex neural ordinary differential equations. *Computer Methods in Applied Mechanics and Engineering*, 398: 115248, 2022a. doi: <https://doi.org/10.1016/j.cma.2022.115248>.

- Vahidullah Taç, Vivek D Sree, Manuel K Rausch, and Adrian B Tepole. Data-driven modeling of the mechanical behavior of anisotropic soft biological tissue. *Engineering with Computers*, 38(5):4167–4182, 2022b. doi: <https://doi.org/10.1007/s00366-022-01733-3>.
- Vahidullah Taç, Manuel K Rausch, Francisco Sahli Costabal, and Adrian Buganza Tepole. Data-driven anisotropic finite viscoelasticity using neural ordinary differential equations. *Computer methods in applied mechanics and engineering*, 411:116046, 2023. doi: <https://doi.org/10.1016/j.cma.2023.116046>.
- Marius Tacke, Matthias Busch, Kian Abdolazizi, Jonas Eichinger, Kevin Linka, Christian Cyron, and Roland Aydin. Automating modeling in mechanics: Llms as designers of physics-constrained neural networks for constitutive modeling of materials. *arXiv preprint arXiv:2512.01735*, 2025a.
- Marius Tacke, Matthias Busch, Kartik Bali, Kian Abdolazizi, Kevin Linka, Christian Cyron, and Roland Aydin. Constitutive scientific generative agent (csga): Leveraging large language models for automated constitutive model discovery. *Machine Learning for Computational Science and Engineering*, 1(1):23, 2025b. doi: <https://doi.org/10.1007/s44379-025-00022-2>.
- Jiabin Tang, Tianyu Fan, and Chao Huang. Autoagent: A fully-automated and zero-code framework for llm agents. *arXiv preprint arXiv:2502.05957*, 2025. doi: <https://doi.org/10.48550/arXiv.2502.05957>.
- Yingheng Tang, Wenbin Xu, Jie Cao, Weilu Gao, Steven Farrell, Benjamin Erichson, Michael W Mahoney, Andy Nonaka, and Zhi Jackie Yao. A multimodal large language model for materials science. *Nature Machine Intelligence*, pp. 1–14, 2026.
- Bilge Taskin, Wenxiong Xie, and Teddy Lazebnik. Knowledge integration for physics-informed symbolic regression using pre-trained large language models. *Scientific Reports*, 2026.
- Prakash Thakolkaran, Akshay Joshi, Yiwen Zheng, Moritz Flaschel, Laura De Lorenzis, and Siddhant Kumar. Nn-euclid: Deep-learning hyperelasticity without stress data. *Journal of the Mechanics and Physics of Solids*, 169:105076, 2022. doi: <https://doi.org/10.1016/j.jmps.2022.105076>.
- LRG Treloar. Stress-strain data for vulcanised rubber under various types of deformation. *Transactions of the Faraday Society*, 40:59–70, 1944. doi: <https://doi.org/10.1039/TF9444000059>.
- LRG Treloar. *The Physics of Rubber Elasticity*. Oxford University Press, Oxford, 10 2005. doi: <https://doi.org/10.1093/oso/9780198570271.001.0001>.
- Tushar, Sawan Kumar, and Souvik Chakraborty. Fusion-based constitutive model (fuce): Toward model-data augmentation in constitutive modeling. *International Journal of Mechanical System Dynamics*, 5(1):86–100, 2025. doi: <https://doi.org/10.1002/msd2.70005>.
- Kshitiz Upadhyay, Ghatu Subhash, and Douglas Spearot. Thermodynamics-based stability criteria for constitutive equations of isotropic hyperelastic solids. *Journal of the Mechanics and Physics of Solids*, 124: 115–142, 2019.
- Kirk C Valanis and Robert F Landel. The strain-energy function of a hyperelastic material in terms of the extension ratios. *Journal of Applied Physics*, 38(7):2997–3002, 1967.
- Samidha Verma, Arushi Goyal, Ananya Mathur, Ankit Anand, and Sayan Ranu. Grail: Graph edit distance and node alignment using llm-generated code. *arXiv preprint arXiv:2505.02124*, 2025. doi: <https://doi.org/10.48550/arXiv.2505.02124>.
- Nikolaos N Vlassis and WaiChing Sun. Sobolev training of thermodynamic-informed neural networks for interpretable elasto-plasticity models with level set hardening. *Computer Methods in Applied Mechanics and Engineering*, 377:113695, 2021.
- Runxiang Wang, Boxiao Wang, Kai Li, Yifan Zhang, and Jian Cheng. Drsr: Llm based scientific equation discovery with dual reasoning from data and experience. *arXiv preprint arXiv:2506.04282*, 2025a.
- Yaoliang Wang, Zhiyong Wu, Junfeng Yao, and Jinsong Su. Tdag: A multi-agent framework based on dynamic task decomposition and agent generation. *Neural Networks*, 185:107200, 2025b.
- Simon Wiesheier, Miguel Angel Moreno-Mateos, and Paul Steinmann. Versatile data-adaptive hyperelastic energy functions for soft materials. *Computer Methods in Applied Mechanics and Engineering*, 430:117208, 2024.
- Yue Wu, Tianhao Su, Mingchuan Zhao, Shunbo Hu, and Deng Pan. Engineering-oriented symbolic regression: Llms as physics agents for discovery of simulation-ready constitutive laws. *arXiv preprint arXiv:2603.19241*, 2026.

- Qingpo Wuwu, Chonghan Gao, Tianyu Chen, Yihang Huang, Yuekai Zhang, Jianing Wang, Jianxin Li, Haoyi Zhou, and Shanghang Zhang. PINNsagent: Automated PDE surrogation with large language models. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=R050G0zs6M>.
- Oskar Wysocki, Danilo Carvalho, Alex Bogatu, André Freitas, et al. An llm-based knowledge synthesis and scientific reasoning framework for biomedical discovery. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pp. 355–364, 2024.
- Shijie Xia, Yuhan Sun, and Pengfei Liu. Sr-scientist: Scientific equation discovery with agentic ai. *arXiv preprint arXiv:2510.11661*, 2025.
- Hongyi Xu, Funshing Sin, Yufeng Zhu, and Jernej Barbič. Nonlinear material design using principal stretches. *ACM Transactions on Graphics (TOG)*, 34(4):1–11, 2015.
- Jianke Yang, Ohm Venkatachalam, Mohammad Kianezhad, Sharvaree Vadgama, and Rose Yu. Think like a scientist: Physics-guided llm agent for equation discovery. *arXiv preprint arXiv:2602.12259*, 2026.
- Zonglin Yang, Wanhao Liu, Ben Gao, Tong Xie, Yuqiang Li, Wanli Ouyang, Soujanya Poria, Erik Cambria, and Dongzhan Zhou. MOOSE-chem: Large language models for rediscovering unseen chemistry scientific hypotheses. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, and Deqing Yang. Evoagent: Towards automatic multi-agent generation via evolutionary algorithms. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 6192–6217, 2025. doi: <https://doi.org/10.18653/v1/2025.naacl-long.315>.
- Layne Zee and Eli Sternberg. Ordinary and strong ellipticity in the equilibrium theory of incompressible hyperelastic solids. *Archive for Rational Mechanics and Analysis*, 83(1):53–90, 1983.
- Hengzhe Zhang, Qi Chen, Wolfgang Banzhaf, Mengjie Zhang, et al. Rag-sr: Retrieval-augmented generation for neural symbolic regression. In *The Thirteenth International Conference on Learning Representations*, 2025.

A Numerical validation of the physical constraints

This appendix specifies the numerical validators described in Section 2.3 in sufficient detail for an independent reimplementaion. All validators receive a trained model and return a binary verdict (passed or failed). A constraint is reported as passed only when no violation is found across the full sampling space. Most validators perform element-wise comparisons through a common helper that combines a relative tolerance of $\tau_{\text{rel}} = 10^{-3}$ — i.e., agreement to three significant digits of the larger argument — with an absolute floor of $\tau_{\text{abs}} = 10^{-4}$ that takes over for values near zero, where any relative tolerance would collapse to floating-point noise. A few simpler validators (energy and stress normalization, non-negativity of the strain energy) instead apply a plain absolute tolerance of $\tau = 10^{-3}$. These choices are a deliberate compromise between sensitivity to genuine violations and robustness to numerical noise.

A.1 Common sampling spaces

Plane-strain deformation gradients. Because the materials considered here are incompressible, all admissible deformation gradients satisfy $\det \mathbf{F} = 1$. We work with plane-strain deformations of the form

$$\mathbf{F} = \begin{pmatrix} F_{11} & F_{12} & 0 \\ F_{21} & F_{22} & 0 \\ 0 & 0 & 1/(F_{11}F_{22} - F_{12}F_{21}) \end{pmatrix}, \quad (1)$$

which guarantees $\det \mathbf{F} = 1$ and matches the experimental loading scenarios (uniaxial, biaxial, pure and simple shear). The base sampling set $\mathcal{F}_{\text{base}}$ contains the identity, uniaxial tension and compression states with stretches $\lambda \in \{0.5, 0.7, 0.9, 1.1, 1.3, 1.5, 2.0, 3.0\}$, equibiaxial states with $\lambda \in \{0.7, 0.9, 1.1, 1.3, 1.5, 2.0\}$, planar (pure) shear states with $\lambda \in \{0.5, 0.7, 0.9, 1.1, 1.5, 2.0, 3.0\}$, simple shear states with $\gamma \in \{-1.0, -0.5, -0.1, 0.1, 0.5, 1.0\}$, off-equibiaxial combinations, and a few combined stretch–shear states. In total, $|\mathcal{F}_{\text{base}}| \approx 35$ representative states.

Three-dimensional rotations. Out-of-plane rotations are sampled as $\mathbf{Q} = \mathbf{R}_x(\varphi_1)\mathbf{R}_y(\varphi_2)\mathbf{R}_z(\varphi_3)$, where $\mathbf{R}_x, \mathbf{R}_y, \mathbf{R}_z$ denote the elementary rotations about the coordinate axes. We use a fixed set Φ of nine angle triples chosen to combine simple multiples of $\pi/2, \pi/3, \pi/4, \pi/6$ with non-rational angles for generality. Reflections are obtained by post-multiplying with $\text{diag}(1, 1, -1)$, yielding the set $\mathcal{Q}^- = \{\mathbf{Q} \text{diag}(1, 1, -1) : \mathbf{Q} \in \mathcal{Q}^+\}$ for material-symmetry tests. In-plane rotations $\mathbf{Q}_{2D}(\theta)$ around the z -axis are used as waypoint constructions in the thermodynamic-consistency loops.

A.2 Thermodynamic consistency

Following the path-independence approach of Upadhyay et al. (2019); González et al. (2019), we test thermodynamic consistency by evaluating four metrics: two on closed deformation loops, and two on a pair of open paths that share the same end states.

Paths. Two closed loops and one two-path comparison are constructed by piecewise-linear interpolation between waypoints in $(F_{11}, F_{12}, F_{21}, F_{22})$ space, with the out-of-plane component recomputed at each step to enforce $\det \mathbf{F} = 1$. Each segment is discretized into $n_{\text{seg}} = 200$ steps.

- Loop 1 uses purely diagonal waypoints and traverses uniaxial tension, planar tension, and equibiaxial states before returning to the identity.
- Loop 2 additionally activates off-diagonal entries through simple shear and includes a waypoint obtained by applying the in-plane rotation $\mathbf{Q}_{2D}(\pi/4)$ to a stretch–shear state, exercising the full chain rule through the invariants.
- Path A connects $\mathbf{I}$ to a uniaxial tension state directly. Path B connects the same end points by first loading through pure shear and then adjusting the transverse stretch at constant axial stretch.

Metrics. At each discretization step k , the model returns the energy Ψ_k and the stress $\mathbf{P}_k$. The incremental and cumulative work are computed via the trapezoidal rule,

$$\delta W_k = \frac{1}{2}(\mathbf{P}_k + \mathbf{P}_{k+1}) : \Delta \mathbf{F}_k, \quad W_k = \sum_{j=0}^{k-1} \delta W_j. \quad (2)$$

On each *closed loop*, two checks are performed:

1. *Normalized loop residual.* The net work over the loop must vanish relative to the total work expended,

$$\eta = |W_N| / \sum_k |\delta W_k| \leq \tau_{\text{loop}}, \quad \tau_{\text{loop}} = 10^{-2}. \quad (3)$$

The tolerance here is looser than the one used elsewhere because trapezoidal integration accumulates discretization error along the full path length, so a stricter bound would conflate genuine thermodynamic violations with quadrature noise.

2. *Stress uniqueness at loop closure.* The model must return to the same stress when the deformation returns to its starting point,

$$\mathbf{P}(\mathbf{F}_{\text{start}}) \approx \mathbf{P}(\mathbf{F}_{\text{end}}), \quad (4)$$

componentwise to relative tolerance τ_{rel} with absolute floor τ_{abs} . Being a direct model evaluation rather than an integrated quantity, this check is independent of any quadrature drift and specifically catches hidden statefulness in the implementation.

On the *two-path comparison*, two further checks are performed:

3. *Two-path agreement at the common end state.* The cumulative work along Path A and Path B must coincide at the shared end state, $W_N^A \approx W_N^B$, to relative tolerance τ_{rel} with absolute floor τ_{abs} .
4. *Pointwise work–energy consistency.* Along each open path, the cumulative work must reproduce the energy difference at every step,

$$|W_k - (\Psi_k - \Psi_0)| \leq \tau_{\text{rel}} \max(|W_k|, |\Psi_k - \Psi_0|), \quad \tau_{\text{rel}} = 10^{-2}. \quad (5)$$

The validator returns “passed” only if all four metrics are satisfied for both loops and for the two-path pair.

A.3 Stress symmetry

For every $\mathbf{F} \in \mathcal{F}_{\text{base}}$ the model is queried for the predicted stress $\mathbf{P}$, the Cauchy stress is computed as $\boldsymbol{\sigma} = \mathbf{P}\mathbf{F}^T$ (using $J = 1$), and the condition $\boldsymbol{\sigma} = \boldsymbol{\sigma}^T$ is verified componentwise to relative tolerance τ_{rel} with absolute floor τ_{abs} for every sampled $\mathbf{F}$.

A.4 Objectivity

We verify

$$\Psi(\mathbf{Q}\mathbf{F}) = \Psi(\mathbf{F}) \quad \forall \mathbf{F} \in \mathcal{F}_{\text{base}}, \forall \mathbf{Q} \in \mathcal{Q}^+, \quad (6)$$

to relative tolerance τ_{rel} with absolute floor τ_{abs} . Since Ψ is computed independently of the Lagrange-multiplier pressure p , the plane-stress assumption $P_{33} = 0$ does not enter this check, and arbitrary three-dimensional rotations $\mathbf{Q} \in \mathcal{Q}^+$ can be used.

A.5 Material symmetry

For isotropic materials, the symmetry group is $\mathcal{O}(3)$. The validator therefore checks

$$\Psi(\mathbf{F}\mathbf{Q}^T) = \Psi(\mathbf{F}) \quad \forall \mathbf{F} \in \mathcal{F}_{\text{base}}, \forall \mathbf{Q} \in \mathcal{Q}^+ \cup \mathcal{Q}^-, \quad (7)$$

to relative tolerance τ_{rel} with absolute floor τ_{abs} , again using the fact that Ψ does not depend on p .

A.6 Ellipticity

We verify the rank-one convexity (Legendre–Hadamard) condition

$$(\mathbf{a} \otimes \mathbf{b}) : \mathbb{A}(\mathbf{F}) : (\mathbf{a} \otimes \mathbf{b}) \geq 0 \quad \forall \mathbf{a}, \mathbf{b} \in \mathbb{R}^3, \quad (8)$$

where $\mathbb{A}_{ijkl}(\mathbf{F}) = \partial^2 \Psi / \partial F_{ij} \partial F_{kl}$ is the fourth-order elasticity tensor, evaluated at admissible deformations $\mathbf{F}$. As argued in Linden et al. (2023); Neff et al. (2015); Zee & Sternberg (1983), ellipticity is the practically relevant implication of polyconvexity, and we test it directly because polyconvexity itself is difficult to validate numerically.

Sampling of $\mathbf{F}$. For isotropic, objective potentials, $\Psi(\mathbf{Q}_1 \mathbf{F} \mathbf{Q}_2^T) = \Psi(\mathbf{F})$ for all $\mathbf{Q}_1 \in \mathcal{SO}(3)$, $\mathbf{Q}_2 \in \mathcal{O}(3)$. The rank-one condition at a general $\mathbf{F}$ with directions $(\mathbf{a}, \mathbf{b})$ is therefore equivalent to the same condition at $\mathbf{F} = \text{diag}(\lambda_1, \lambda_2, \lambda_3)$ with appropriately rotated directions $(\mathbf{Q}_1 \mathbf{a}, \mathbf{Q}_2 \mathbf{b})$, which still range over all of $\mathbb{R}^3$. Incompressibility further restricts the principal stretches to the surface $\lambda_1 \lambda_2 \lambda_3 = 1$, and isotropy makes the condition invariant under permutations of $(\lambda_1, \lambda_2, \lambda_3)$. We therefore sample on the ordered sector $\lambda_1 \geq \lambda_2 \geq \lambda_3$ with $\lambda_3 = (\lambda_1 \lambda_2)^{-1}$. The principal stretches are placed on a regular grid in logarithmic stretch space,

$$\log \lambda_1, \log \lambda_2 \in [-L, L], \quad L = 2, \quad (9)$$

with 50×50 grid points, so that extension and compression are treated symmetrically. After ordering, deduplication, and discarding states with $\lambda_3 \leq e^{-3L}$, this yields the validation set $\mathcal{F}_{\text{ell}}$.

Sampling of $\mathbf{a}$ and $\mathbf{b}$. Equation (8) is homogeneous of degree two in both $\mathbf{a}$ and $\mathbf{b}$, so it is sufficient to sample unit vectors. Furthermore, the substitution $\mathbf{a} \mapsto -\mathbf{a}$ leaves the form invariant, so each direction can be restricted independently to a hemisphere. We use a Fibonacci spiral with $n_{\text{dirs}} = 200$ pre-mask samples to obtain near-uniform coverage of the upper hemisphere, generating two independent sets $\mathcal{A}$ and $\mathcal{B}$, and then build all pairs $(\mathbf{a}, \mathbf{b}) \in \mathcal{A} \times \mathcal{B}$. As emphasized in Neff et al. (2015); Zee & Sternberg (1983), axis-aligned direction pairs probe only the necessary Baker–Ericksen and tension–extension inequalities; the genuinely three-dimensional rank-one condition is exposed only by oblique pairs, which the Fibonacci sampling provides.

Evaluation. For each $\mathbf{F} \in \mathcal{F}_{\text{ell}}$, the elasticity tensor $\mathbb{A}(\mathbf{F})$ is computed by nested automatic differentiation of Ψ with respect to $\mathbf{F}$ and reshaped into a 9×9 matrix $\mathbb{A}_{\text{flat}}$. For each direction pair, the scalar

$$\mathcal{R}(\mathbf{F}, \mathbf{a}, \mathbf{b}) = \mathbf{h}^T \mathbb{A}_{\text{flat}}(\mathbf{F}) \mathbf{h}, \quad \mathbf{h} = \text{vec}(\mathbf{a} \otimes \mathbf{b}), \quad (10)$$

is evaluated. The deformation set is processed in mini-batches of 64 for memory efficiency. The validator returns “passed” if $\mathcal{R} \geq 0$ across the full sample, and “failed” as soon as a negative value is encountered.

A.7 Growth condition

Under the incompressibility constraint, every admissible deformation satisfies $J = 1$, and the growth condition $\Psi \rightarrow \infty$ as $J \rightarrow 0^+$ or $J \rightarrow \infty$ is therefore trivially fulfilled on the admissible set. The validator returns “passed” without any model evaluation.

A.8 Energy normalization

The model is queried at $\mathbf{F} = \mathbf{I}$, and the validator checks

$$|\Psi(\mathbf{I})| \leq \tau. \quad (11)$$

A.9 Stress normalization

Analogously, the model is queried at $\mathbf{F} = \mathbf{I}$, and the validator checks

$$\|\mathbf{P}(\mathbf{I})\|_{\infty} \leq \tau. \quad (12)$$

A.10 Non-negativity of the strain energy

Finally, the validator evaluates Ψ at every $\mathbf{F} \in \mathcal{F}_{\text{base}}$ and checks

$$\Psi(\mathbf{F}) \geq -\tau \quad \forall \mathbf{F} \in \mathcal{F}_{\text{base}}. \quad (13)$$

A small negative tolerance is admitted to absorb floating-point noise around $\Psi = 0$ in the reference configuration.

B Adherence of generated models to physical constraints in absolute numbers

Figure 4 in Section 4.1 reports the adherence of the generated models to the physical constraints in relative terms, which is the most informative view for judging how much a user can trust a model returned by the pipeline. For completeness, we also provide the underlying absolute counts in Figure 13. Two points are worth noting when interpreting these numbers. First, every refinement round exports exactly one inspector-approved model, so the totals on the flagged-as-adhering side directly reflect the number of runs. Second, models flagged by the inspector as violating are not exported by default; we export only a random subset of them for the purpose of this analysis, which is why the totals on the flagged-as-violating side are substantially lower. This imbalance is a consequence of our experimental design and does not affect the relative frequencies discussed in Section 4.1.

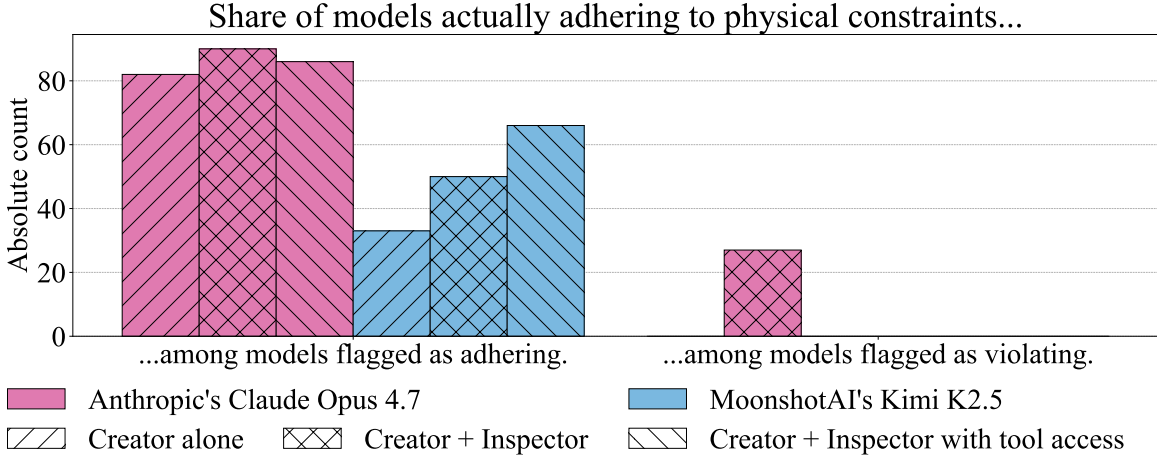


Figure 13: Adherence of generated models to physical constraints in absolute counts. Same layout as Figure 4, but reporting absolute numbers of models rather than relative frequencies. The lower counts on the flagged-as-violating side reflect that only a random subset of violating models is exported for analysis, whereas every refinement round exports exactly one inspector-approved model.

C Model implementations generated for the experimental rubber dataset

To give a concrete impression of the artifacts produced by our pipeline, we show below the Python implementations of the two CANNs whose predictions on the experimental rubber dataset are shown in Figures 8 and 9. Both were generated by the Creator-Inspector pipeline without tool access and correspond, in each case, to the most accurate adherent model out of ten independent runs — the Opus-powered pair in Listing 1 and the Kimi-powered pair in Listing 2.

Listing 1: Model implementation for the experimental rubber dataset generated by the Creator-Inspector pipeline powered by Claude Opus 4.7 (best-out-of-ten adherent run, no tool access).

```

1
2
3 import tensorflow as tf
4
5
6 class NonNegative(tf.keras.constraints.Constraint):
7     def __call__(self, w):
8         return tf.nn.relu(w)
9
10
11 class CANNBlock(tf.keras.layers.Layer):
12     """
13     Polyconvex building block.
14
15     Input:  $x = (\text{invariant} - 3) \geq 0$ , shape (batch, 1).
16     Output: non-negative scalar  $\psi$  contribution, shape (batch, 1).
17
18     All activations are:
19     * convex,
20     * monotonically increasing,
21     *  $C^2$  (or smoother),
22     * zero at  $x = 0$  (so  $\psi(C=I)=0$ ; explicit subtraction also applied).
23
24     Neurons per activation type, each with its own non-negative inner and
25     outer weights. The inner weights act inside the nonlinearity; the outer
26     weights scale the result.
27     """
28
29     def __init__(self, num_neurons=2, **kwargs):
30         super().__init__(**kwargs)
31         self.num_neurons = num_neurons
32
33     def build(self, input_shape):
34         n = self.num_neurons
35
36         # Modest initializers to avoid blow-ups; spread scales over neurons.
37         init_in_s = tf.keras.initializers.RandomUniform(0.05, 0.3)
38         init_in_m = tf.keras.initializers.RandomUniform(0.1, 0.5)
39         init_out = tf.keras.initializers.RandomUniform(0.02, 0.15)
40         init_out_small = tf.keras.initializers.RandomUniform(1e-3, 2e-2)
41
42         # --- Linear:  $f(y) = y$  (Neo-Hookean-like) ---
43         self.w_in_lin = self.add_weight(
44             name="w_in_lin", shape=(n,), initializer=init_in_s,
45             constraint=NonNegative(), trainable=True)
46         self.w_out_lin = self.add_weight(
47             name="w_out_lin", shape=(n,), initializer=init_out,
48             constraint=NonNegative(), trainable=True)
49
50         # --- Quadratic:  $f(y) = y^2$  ---
51         self.w_in_sq = self.add_weight(
52             name="w_in_sq", shape=(n,), initializer=init_in_s,

```

```

53         constraint=NonNegative(), trainable=True)
54     self.w_out_sq = self.add_weight(
55         name="w_out_sq", shape=(n,), initializer=init_out_small,
56         constraint=NonNegative(), trainable=True)
57
58     # --- Softplus-based:  $f(y) = \text{softplus}(y) - \log(2)$  ---
59     #  $f(0)=0$ ,  $f'(0)=1/2>0$ ,  $f''\geq 0$ , smooth.
60     self.w_in_sp = self.add_weight(
61         name="w_in_sp", shape=(n,), initializer=init_in_m,
62         constraint=NonNegative(), trainable=True)
63     self.w_out_sp = self.add_weight(
64         name="w_out_sp", shape=(n,), initializer=init_out,
65         constraint=NonNegative(), trainable=True)
66
67     # --- Squared softplus:  $f(y) = (\text{softplus}(y) - \log(2))^2$  ---
68     # Convex, monotonically increasing for  $y\geq 0$ , smooth,  $f(0)=0$ .
69     self.w_in_sp2 = self.add_weight(
70         name="w_in_sp2", shape=(n,), initializer=init_in_m,
71         constraint=NonNegative(), trainable=True)
72     self.w_out_sp2 = self.add_weight(
73         name="w_out_sp2", shape=(n,), initializer=init_out_small,
74         constraint=NonNegative(), trainable=True)
75
76     # --- Exponential-like (Demiray/Holzapfel):  $f(y) = \exp(y) - 1 - y$  ---
77     #  $f(0)=0$ ,  $f'(y)=\exp(y)-1\geq 0$  for  $y\geq 0$ ,  $f''(y)=\exp(y)>0$ , smooth.
78     self.w_in_exp = self.add_weight(
79         name="w_in_exp", shape=(n,),
80         initializer=tf.keras.initializers.RandomUniform(0.05, 0.25),
81         constraint=NonNegative(), trainable=True)
82     self.w_out_exp = self.add_weight(
83         name="w_out_exp", shape=(n,),
84         initializer=tf.keras.initializers.RandomUniform(1e-4, 5e-3),
85         constraint=NonNegative(), trainable=True)
86
87     super().build(input_shape)
88
89     def call(self, x):
90         # x shape: (batch, 1). Weight shape: (n,). Broadcast -> (batch, n).
91
92         # Linear
93         y_lin = self.w_in_lin * x
94         a_lin = y_lin
95         out_lin = tf.reduce_sum(self.w_out_lin * a_lin, axis=-1, keepdims=True)
96
97         # Quadratic
98         y_sq = self.w_in_sq * x
99         a_sq = tf.square(y_sq)
100        out_sq = tf.reduce_sum(self.w_out_sq * a_sq, axis=-1, keepdims=True)
101
102        # Softplus shifted
103        y_sp = self.w_in_sp * x
104        # Limit argument to avoid overflow (softplus is linear for large y).
105        y_sp = tf.clip_by_value(y_sp, 0.0, 30.0)
106        a_sp = tf.nn.softplus(y_sp) - tf.math.log(2.0)
107        out_sp = tf.reduce_sum(self.w_out_sp * a_sp, axis=-1, keepdims=True)
108
109        # Squared softplus
110        y_sp2 = self.w_in_sp2 * x
111        y_sp2 = tf.clip_by_value(y_sp2, 0.0, 30.0)
112        a_sp2 = tf.square(tf.nn.softplus(y_sp2) - tf.math.log(2.0))
113        out_sp2 = tf.reduce_sum(self.w_out_sp2 * a_sp2, axis=-1, keepdims=True)
114
115        # Exponential-like
116        y_exp = self.w_in_exp * x
117        # Strict clipping to keep things numerically safe.

```

```

118     y_exp = tf.clip_by_value(y_exp, 0.0, 10.0)
119     a_exp = tf.exp(y_exp) - 1.0 - y_exp
120     out_exp = tf.reduce_sum(self.w_out_exp * a_exp, axis=-1, keepdims=True)
121
122     return out_lin + out_sq + out_sp + out_sp2 + out_exp
123
124
125 class CANN(tf.keras.Model):
126     def __init__(self, num_neurons=2, **kwargs):
127         super().__init__(**kwargs)
128         self.block_I1 = CANNBlock(num_neurons=num_neurons, name="block_I1")
129         self.block_I2 = CANNBlock(num_neurons=num_neurons, name="block_I2")
130
131     def psi_from_F(self, F):
132         # Right Cauchy-Green tensor
133         C = tf.matmul(F, F, transpose_a=True)
134         I1 = tf.linalg.trace(C)
135         C2 = tf.matmul(C, C)
136         I2 = 0.5 * (tf.square(I1) - tf.linalg.trace(C2))
137
138         I1 = tf.reshape(I1, (-1, 1))
139         I2 = tf.reshape(I2, (-1, 1))
140
141         # For incompressible isotropic materials I1, I2 >= 3. Clamp noise.
142         x1 = tf.nn.relu(I1 - 3.0)
143         x2 = tf.nn.relu(I2 - 3.0)
144
145         psi_raw = self.block_I1(x1) + self.block_I2(x2)
146
147         # Explicit energy normalization: psi(C=I) = 0.
148         zero_in = tf.zeros_like(x1)
149         psi0 = self.block_I1(zero_in) + self.block_I2(zero_in)
150         psi = psi_raw - psi0
151         return psi
152
153     def call(self, F):
154         with tf.GradientTape() as tape:
155             tape.watch(F)
156             psi = self.psi_from_F(F)
157             dpsi_dF = tape.gradient(psi, F)
158
159             # P = dpsi/dF - p * F^{-T}, enforcing P[2,2] = 0 (plane-stress-like in
160             # the 3-direction). This determines the Lagrange multiplier p.
161             F_invT = tf.linalg.inv(tf.linalg.matrix_transpose(F))
162             num = dpsi_dF[:, 2, 2]
163             den = F_invT[:, 2, 2]
164             den_safe = tf.where(tf.abs(den) < 1e-8,
165                                 tf.ones_like(den) * 1e-8, den)
166             p = num / den_safe
167             p = tf.reshape(p, (-1, 1, 1))
168
169             P = dpsi_dF - p * F_invT
170             return {"P": P, "Psi": psi}
171
172
173     def build_cann_model():
174         model = CANN(num_neurons=2)
175         # Force a build with a dummy input so all weights are created.
176         dummy_F = tf.eye(3, batch_shape=[1], dtype=tf.float32)
177         _ = model(dummy_F)
178         return model

```

Listing 2: Model implementation for the experimental rubber dataset generated by the Creator–Inspector pipeline powered by Kimi K2.5 (best-out-of-ten adherent run, no tool access).

```

1
2
3 import tensorflow as tf
4
5
6 class CANN(tf.keras.Model):
7     def __init__(self, **kwargs):
8         """
9         Args:
10             kwargs: Additional keyword arguments for the model initialization.
11         """
12         super(CANN, self).__init__(**kwargs)
13
14         # Define a convex neural network architecture
15         # Using non-negative weights and convex, monotonically increasing activations (
16         Softplus)
17         hidden_units = [10, 10] # Default architecture, can be adjusted
18
19         self.hidden_layers = []
20         for units in hidden_units:
21             self.hidden_layers.append(
22                 tf.keras.layers.Dense(
23                     units,
24                     activation='softplus',
25                     kernel_constraint=tf.keras.constraints.NonNeg(),
26                     dtype='float32'
27                 )
28             )
29
30         # Output layer: scalar strain energy density
31         self.psi_output = tf.keras.layers.Dense(
32             1,
33             activation='softplus',
34             kernel_constraint=tf.keras.constraints.NonNeg(),
35             dtype='float32'
36         )
37
38     def _compute_invariants(self, F):
39         """Compute the first and second invariants of the right Cauchy-Green tensor C."""
40         #  $C = F^T * F$ 
41         C = tf.matmul(F, F, transpose_a=True)
42
43         # First invariant:  $I1 = \text{tr}(C)$ 
44         I1 = tf.linalg.trace(C)
45
46         # Second invariant:  $I2 = 0.5 * (\text{tr}(C)^2 - \text{tr}(C^2))$ 
47         trace_C2 = tf.linalg.trace(tf.matmul(C, C))
48         I2 = 0.5 * (tf.square(I1) - trace_C2)
49
50         # Reshape to (batch_size, 1)
51         I1 = tf.expand_dims(I1, axis=-1)
52         I2 = tf.expand_dims(I2, axis=-1)
53
54         return I1, I2
55
56     def psi_from_F(self, F):
57         """
58         Pure forward computation of psi from F. No tf.GradientTape inside.
59         Args:

```

```

60         F (tf.Tensor): Input tensor containing the deformation gradient F. Shape: (
batch_size,3,3).
61     Returns:
62         tf.Tensor: Strain energy density psi predicted from the deformation gradient F.
Shape: (batch_size,1)
63     """
64     # Compute invariants I1 and I2
65     I1, I2 = self._compute_invariants(F)
66
67     # Prepare input features (batch_size, 2)
68     features = tf.concat([I1, I2], axis=-1)
69
70     # Forward pass through the network
71     x = features
72     for layer in self.hidden_layers:
73         x = layer(x)
74     psi_raw = self.psi_output(x)
75
76     # Energy normalization: subtract psi evaluated at C = I (where I1=3, I2=3)
77     I1_id = tf.constant([[3.0]], dtype=tf.float32)
78     I2_id = tf.constant([[3.0]], dtype=tf.float32)
79     features_id = tf.concat([I1_id, I2_id], axis=-1)
80
81     x_id = features_id
82     for layer in self.hidden_layers:
83         x_id = layer(x_id)
84     psi_id = self.psi_output(x_id)
85
86     psi = psi_raw - psi_id
87
88     return psi
89
90 def call(self, F):
91     """
92     Uses method psi_from_F and tf.GradientTape to predict both psi and P from F.
93
94     Args:
95         F (tf.Tensor): Input tensor containing the deformation gradient F. Shape: (
batch_size,3,3)
96     Returns:
97         dict: Dictionary with keys "P" and "Psi":
98             - "P": First Piola-Kirchhoff stress tensor. Shape: (batch_size,3,3).
99             - "Psi": Strain energy density. Shape: (batch_size,1).
100     """
101     with tf.GradientTape() as tape:
102         tape.watch(F)
103         psi = self.psi_from_F(F)
104
105     # Compute isochoric part of P = psi/F
106     P_iso = tape.gradient(psi, F)
107
108     # Handle potential None gradient (should not happen with valid operations)
109     if P_iso is None:
110         P_iso = tf.zeros_like(F)
111
112     # Compute volumetric part based on hydrostatic pressure p
113     # For incompressible materials: P = P_iso - p * F^{-T}
114     F_inv = tf.linalg.inv(F)
115     F_inv_T = tf.transpose(F_inv, perm=[0, 2, 1])
116
117     # Determine p from boundary condition P_{33} = 0 (index 2,2)
118     # P_{33} = P_iso_{33} - p * (F^{-T})_{33} = 0
119     # => p = P_iso_{33} / (F^{-T})_{33}
120     P_iso_33 = P_iso[:, 2, 2]
121     F_inv_T_33 = F_inv_T[:, 2, 2]

```

```

122
123     # Add small epsilon to avoid division by zero
124     eps = 1e-8
125     p = P_iso_33 / (F_inv_T_33 + eps)
126
127     # Reshape p for broadcasting: (batch_size, 1, 1)
128     p = tf.reshape(p, [-1, 1, 1])
129
130     # Total first Piola-Kirchhoff stress
131     P = P_iso - p * F_inv_T
132
133     return {"P": P, "Psi": psi}
134
135
136 def build_cann_model():
137     """ Builds a CANN model for predicting the strain energy density psi and the first
138     Piola Kirchhoff stress P from the deformation gradient F for hyperelastic,
139     incompressible, isotropic materials.
140     Args:
141         -- None --
142     Returns:
143         CANN: A CANN model instance (subclass of tf.keras.Model).
144     """
145     return CANN()

```
